集成 Ascon-AEAD128 轻量密码加速的 RISC-V 处理器 SoC

队伍编号 TBD　团队名称 TBD

全国大学生嵌入式芯片与系统设计竞赛 2026 · FPGA 创新设计赛道 · AMD 自主选题初级组

2026-10-29

目录

摘要 **1**

第一部分作品概述 **2**

1 功能与特性 2
2 应用领域 3
3 主要技术特点 3
4 主要性能指标 4
5 主要创新点 5
6 设计流程 5
6.1 研制历程 5
6.2 设计准则与证据分级 6
6.3 术语与统计口径 7

第二部分系统组成及功能说明 **9**

1 整体介绍 9
2 硬件系统介绍 10
2.1 硬件整体介绍 10
2.2 机械设计介绍 12
2.3 电路各模块介绍 12
3 软件系统介绍 30
3.1 软件整体介绍 30
3.2 软件各模块介绍 32

第三部分完成情况及性能参数 **34**

1 整体介绍 34
2 工程成果 35
2.1 资源占用 35
2.2 可复现性与产物冻结 36
3 特性成果 37
3.1 时序收敛结果 37
3.2 板级 IO 时序签核 39
3.3 功能验证 40

3.4 性能量化 45
3.5 完成度与判据矩阵 47

第四部分总结 **48**
1 可扩展之处 48
2 心得体会 48

第五部分参考文献 **50**

附录 A 术语与缩写 **51**

附录 B 地址映射与指令编码 **52**
B.1 SoC 地址映射 52
B.2 Ascon 定制指令编码（RV32，custom-0 主操作码） 52
B.3 MMIO 加速器寄存器（节选） 52
B.4 PYNQ AXI-Lite 窗口（PS 侧） 53

附录 C 证据索引 **54**

摘要

本作品面向嵌入式数据安全场景，在 AMD Zynq-7000（XC7Z020CLG400-1，正点原子领航者 ZYNQ7020 开发板）平台上实现了一款集成轻量级密码加速的 RISC-V 处理器 SoC，并建立了涵盖 RTL 仿真、标准测试向量、自动化回归、板级运行与实现签核的可追溯验证链条。处理器为自研五级流水线 RV32IM + Zicsr 软核，包含动态分支预测、指令与数据 Cache、AXI 总线，以及 UART、GPIO、CLINT 等外设，工作频率 100 MHz。

密码侧实现符合 NIST SP 800-232 的 Ascon-AEAD128 认证加密，并给出三种实现方案：纯软件（SW）、存储器映射加速器（MMIO）与定制指令扩展（ISE，即以 custom-0 主操作码下的四条定制指令实现状态读写与 P12/P8 置换）。三者共享同一套状态表示与冻结测试向量，可逐字节交叉比对，用于量化”总线外设式加速”与”定制指令式加速”两条技术路线在周期与实现代价上的实际取舍。板上实测 16 字节加密：软件实现 20806 周期，ISE 实现 2570 周期（周期加速比 8.10×），MMIO 端到端 688 周期（30.24×，其中密码核本体 103 周期）；三套实现的官方测试向量、解密往返与篡改拒绝均通过；三份镜像由同一源集生成，其时序结果均为 WNS +0.087 ns、TNS 0、DRC 0 错误。

验证体系分为四层：模块级 RTL 测试（置换、AEAD、Cache 事务契约、中断抢占）、SoC 级仿真（三套固件 + 官方 KAT + 篡改拒绝）、基于 Icarus Verilog 的自动化回归（smoke 6 项通过、完整回归 15 项通过，单命令输出 JSON/Markdown 报告）与板上验证。固件与 BRAM 初值 COE 由脚本统一生成，RTL、COE、bitstream 与串口原始日志均建立 SHA-256 索引。实现层面，本作品以 33 组内置实现策略扫掠结合三处 RTL 结构改造，将基线 −0.942 ns 的时序违例收敛至正余量，并针对板级 7 个异步 IO 端口建立了可机检的时序签核判据（7 项全部通过）。

在硬件信任根方向，完成了基于 256 路环形振荡器 PUF 与 57 位 Device DNA 的密钥子系统：RO 阵列以独立 pblock 隔离，以 RO 自身为时钟计数，经多次采样多数表决，并通过 Ascon P12 派生 128 位密钥，密钥全程不进入 CPU 可读地址空间。板上实测 enroll 质量 128/128，enroll 与 reconstruct 结果一致，无 helper 时按设计执行 DNA-only 降级路径。后续工作包括 PS/PL 与 Jupyter 交互界面、PUF 跨上电稳定性统计，以及板上 ISA 抽样回归。

在 PS/PL 协同方向，建立了以既有 UART MMIO 为数据通道的 PYNQ overlay：PL 内新增串口桥与控制窗口，不修改冻结 RTL，于板上完成 overlay 加载、协议握手与三模式全容量 KAT。板上在 16 字节容量内对 message 与 AD 各 0 至 16 字节的全部 289 组组合执行，SW、HW、ISE 三种模式的加密 KAT 与错误 Tag 拒绝均为 289/289 通过，服务固件报告 `simulated=false`。

关键词：RISC-V；Ascon-AEAD128；定制指令扩展；存储器映射加速器；RO PUF；Zynq-7000；时序收敛；PYNQ

第一部分作品概述

1 功能与特性

本作品面向”轻量密码算法的可验证硬件加速”这一具体问题，在 Zynq-7020（XC7Z020CLG400-1）上实现了一套可测量、可复现的 RISC-V 片上系统：自研五级流水线 RV32IM + Zicsr 处理器核、标准 Ascon-AEAD128 的三种调用方式（纯软件、存储器映射加速器、定制指令扩展）、基于环形振荡器的物理不可克隆函数（RO-PUF）密钥子系统，以及 PS/PL 协同的 PYNQ 展示通路。四条路径共用同一套指令集、同一套固件结构与同一套 KAT 判据，因而”软件基线—加速器—定制指令”三者的差异可以被单独隔离、单独测量，而不受平台差异干扰。

功能上可分四组。其一，密码功能：Ascon-AEAD128 的初始化、关联数据吸收、加解密与终结全部按 NIST SP 800-232 标准实现，支持 0 至 16 字节明文与 0 至 16 字节关联数据，错误 Tag 一律拒绝。其二，加速功能：MMIO 方案将置换核映射至 `0x3000_0000`，CPU 以寄存器读写驱动；ISE 方案将置换与控制流下沉至 EX 级，以 4 条定制指令（`ascon.w` / `ascon.rd` / `ascon.p12` / `ascon.p8`）取代函数调用。其三，信任根功能：256 路环形振荡器经多次采样与多数表决生成器件指纹，与芯片 DNA 共同派生 320 bit 会话密钥，密钥只存在于 PL 内部通路，不进入 CPU 可读地址空间。其四，展示与测量功能：PS 端 Python 通过两条 AXI-Lite 窗口与 PL 内的服务固件通信，可在 Jupyter 中一键运行三种模式的 KAT 与周期测量，并读回 `mcycle`、CPI 与逐次采样结果。

上述功能均以板上实测数据为判据：同一 16 字节向量下，软件 20 806 周期、MMIO 端到端 688 周期、ISE 2 570 周期；三套镜像的板级 KAT 分别为 6/6、4/4、4/4；PYNQ 通路在 289 组随机向量上三种模式各 289/289 通过，且三次模式切换均未出现错误 Tag 被误接受的情形。

嵌入式设备的数据保护存在一组相互制约的约束：算法须足够轻量，方能在无操作系统、无密码库、主频为数兆至一百兆赫兹的 MCU 级处理器上获得可用吞吐；算法亦须标准化，否则认证加密的正确性无法由第三方复算。Ascon 在 NIST 轻量密码竞赛（LWC）中胜出并纳入 NIST SP 800-232（2025 年 8 月定稿），同时满足上述两项约束：其置换仅有 320 bit 状态，S-box 为 5 bit 布尔函数，适于嵌入处理器数据通路；其 Ascon-AEAD128 的 KAT（known-answer test）公开且可逐字节核对。

据此，本作品将目标界定为：**在自研 RV32 处理器上，将 Ascon-AEAD128 的加速实现为可比较、可复现、可签核的工程闭环**。该目标分解为四条可判定的验收线：

- **标准线**：软件实现与官方 1,089 组 KAT 逐字节一致，且解密失败须拒绝并清除候选明文；
- **加速判据**：在同一 SoC 内实现 MMIO 与 ISE 两种加速通路，给出同板、同频条件下的

周期数与周期加速比；

- **实现线**：100 MHz 下时序收敛（WNS、TNS、失败端点与 DRC 全部通过），并生成可上板镜像；
- **可复现线**：RTL、约束、IP、COE、bitstream 与串口日志全部哈希冻结，可独立复算。

处理器为本作品自研的五级流水线 RV32IM 软核，含指令与数据 Cache、AXI 互连及 UART / GPIO / CLINT 等外设。围绕 Ascon 加速，本作品完成三方面工作：建立 Zynq-7020 板级工程并完成器件与约束适配；增加 MMIO 与定制指令两条加速通路；补齐时序收敛、板级签核与 PS/PL 协同证据。

2 应用领域

本作品的技术形态是” 可替换密码原语的嵌入式 SoC 评估平台”，其可直接迁移的应用场景有四类。

一是资源受限嵌入式设备的对称加密。Ascon 已被 NIST 选为轻量密码标准（SP 800-232），其设计目标正是 8 位与 32 位微控制器、RFID 与传感器节点等算力与存储受限的场合。本作品的 MMIO 与 ISE 两种加速方案，在不改变指令集架构兼容性的前提下把 16 字节 AEAD 的周期开销从 20 806 降至 688 与 2 570，使” 低主频 + 小容量 BRAM” 的 FPGA SoC 也能承载 AEAD 级安全，可用于工业传感器网关、智能表计与资产标签的数据保护。

二是 FPGA 加速器的教学与评测。同一功能以三种实现并列存在，且共用同一固件结构与同一 KAT 判据，使” 软件基线、总线型加速器、指令级加速” 三者的面积、时序与周期代价可以在同一源集、同一板上直接对照。该形态适合作为计算机体系结构、硬件安全与 EDA 课程的实验载体：学生可在既有框架上替换为其他轻量原语（如 GIFT-COFB、Xoodyak），

三是可信启动与密钥管理的预研载体。PUF 子系统给出” 器件指纹 + 芯片 DNA → 会话密钥” 的完整通路，密钥不落存储、不经 CPU 地址空间。该通路可用于 FPGA 加速卡、边缘网关的启动认证与固件加密场景，亦可用作密钥托管方案的原型验证。

四是轻量密码实现的可复现性研究。本作品的每一步结论都绑定源集哈希、run 名称与产物文件（附录 C），周期、面积与时序三个维度的度量口径在文中逐项声明，可直接作为同行复算与横向比较的数据源。

3 主要技术特点

其一，三种实现并列且同源可比。纯软件、MMIO 加速器与定制指令三者共用同一置换核语义、同一固件结构与同一向量集，差异仅限” 调用方式”，因此周期差异可归因于架构而非实现细节。这种” 同一算法、同一平台、三种调用路径” 的对照在同类作品中少见，也使加速比具有明确语义：$S = N_{\mathrm{SW}}/N_X$，分子分母同源同向量。

其二，定制指令按标准机制实现，不私改工具链。4 条指令占用 RISC-V 预留的 custom-0 主操作码空间，以 funct3 区分功能，汇编层使用 `.insn r` 手工编码，未改动 GCC backend。好处是任何标准工具链都能编译该固件，评审可直接用官方工具复现，代价是需在软件侧显式书写指令编码。

其三，时序收敛以”端点归因”而非”策略试错”完成。先以 33 组内置策略并行扫掠确定策略增益上限（回收 0.824 ns，87%），再对剩余 156 个失败端点逐点归因，定位三处结构性瓶颈（置换核复位扇出、乘法器组合深度、Cache miss 路径），改造后由 −0.942 ns 收敛至 +0.087 ns。整个过程保留每阶段的 run 名称与源集哈希。

其四，板级 IO 约束以机理而非经验确定。报告给出四种失败写法与一种交付写法的实测对照，并归纳出”input/output delay 大于捕获时钟周期时 requirement 塌缩” 这一可推广的机理，是同类设计中反复出现却少有成文解释的问题。

其五，验证与度量按四层分级并逐项留证。主机 KAT、RTL 单元仿真、SoC 自动回归、报告中的每个数值都可回溯到具体文件（附录 C）。

4 主要性能指标

下表给出本作品的核心量化指标，全部为实测值。周期类数据取自板上固件的 `mcycle` 读数（16 字节明文、无关联数据、warm 路径、100 MHz、同一固件骨架），时序与资源类数据取自 Vivado 2023.2 在三镜像源集上的最终实现报告，功能类数据取自板级 KAT 与 PYNQ 通路实测。三者源集不同，故分列而不合并，详见 §1.4 与 §3.4。

类别	指标	数值	口径
周期	纯软件 AEAD-128 (16 B)	20 806 周期	板上 `mcycle`
周期	MMIO 端到端	688 周期	板上 `mcycle`
周期	MMIO 核本体	103 周期	状态读写之外的置换本体
周期	定制指令 ISE	2 570 周期	板上 `mcycle`
加速比	MMIO 端到端 / 核本体	8.10× / 202×	对软件基线
加速比	定制指令 ISE	30.24×	对软件基线
时序	三镜像最终 WNS / TNS	+0.087 ns / 0	55 572 端点，WHS +0.021 ns
时序	PYNQ overlay WNS	+0.034 ns	62 861 端点，DRC 0/0
资源	LUT / FF / BRAM / DSP	12 426（23.4%）/ 16 156（15.2%）/ 34.5（24.6%）/ 4（1.8%）	placed 报告
功能	板级 KAT	SW 6/6、ISE 4/4、MMIO 4/4	三套镜像
功能	主机 KAT	1 089/1 089	参考实现互校
功能	PYNQ 通路	三种模式各 289/289，Tag 拒绝各 289/289	289 组随机向量
安全	PUF 采样	256 路 RO，25 MHz 自时钟，约 3.3 ms	5 次采样多数表决

需要说明的边界：表中 +0.087 ns 属芯片内部时序，不含板级 IO；面积对比仅在” 同一源集、同一策略、同一版本工具” 下成立，不与厂商 IP 或不同器件横向比较；软件基线为自研

RV32IM 核上的实现，非 x86/ARM 平台，故 8.10× 与 30.24× 不应外推为跨平台结论。

5 主要创新点

下表汇总本作品的创新点及其对应的可验证证据。创新点的判据不是” 想法新颖”，而是” 在仓库中有对应实现、在板上有对应测量”。

序号	创新点	关键实现	可验证证据
1	同一算法三种调用方式同源对照	共用置换核与固件骨架，仅调用方式不同	§3.4 周期表；§2.3.10–§2.3.14
2	定制指令按标准预留编码实现，不改工具链	custom-0 + `funct3`，`.insn r` 手工编码	§2.3.13；反汇编与执行轨迹
3	以端点归因而非策略试错完成时序收敛	33 组策略扫掠 + 三处结构改造	§3.1；WNS $-0.942 \rightarrow +0.087$
4	以机理确定板级 IO 约束，并给出四种失败写法对照	`false_path` + 形式 `delay` 并用	§3.2；7 条机检判据 PASS

其中第 3、4 两项属方法层面的贡献：它们不改变功能，却决定” 能否在给定器件上收敛并签核”，且所归纳的机理可推广至其他 FPGA SoC 设计。

6 设计流程

本作品的设计流程以” 先确立度量口径，再设计实现，最后留证冻结” 为主线，共六个阶段（图 1-1）。其一，以官方模板与选题要求确定成文结构与指标项，明确每个数值的出处与口径；其二，按模块划分完成 RTL 设计，每个模块同步编写 testbench，避免” 先写完后补测”；其三，以主机参考实现与官方向量建立 L0 期望值，再以 L1 单元仿真、L2 SoC 回归逐层向上收敛；其四，综合与实现阶段先以策略扫掠确定工具增益上限，再按失败端点归因做结构改造，不做无依据的策略堆叠；其五，时序与 IO 签核通过后烧录三套镜像，逐份复验 KAT 并采集周期；其六，冻结源集哈希、bitstream 与报告，保证任一结论可被他人复算。

6.1 研制历程

阶段	时间	关键动作与结果
平台迁移与工程建立	2026-09	整理早期 Vivado 工程（XCI/XDC/COE）与历史资料；确认原 XPR 的 part 实为 `xc7k325t`，与提交板卡不符，据此另建 Zynq 工程；RTL 逐文件比对以确认移植前后等价
标准锚定	2026-10-07	锁定 NIST SP 800-232 Final 与 ascon-c、pyascon 固定 commit；独立下载 1,089 组 KAT；静态审计定位 RTL 七项缺陷（R01–R07）

阶段	时间	关键动作与结果
三实现落地	2026-10-09 至 10-16	standalone AEAD RTL 改为显式长度接口；新增 MMIO 窗口与 ZEROIZE 机制；ISE 接入 EX 级 hold/cancel；SW、HW、ISE 三套固件于 RV32 仿真运行通过
时序收敛	2026-10-17 至 10-20	33 组策略扫掠（WNS $-0.942 \rightarrow -0.118$ ns）；A/C/B 三处 RTL 结构改造（`ascon_ise` 局部复位寄存器化、乘法器输入输出寄存器化、store 与 MMIO 绕过 `MISS_CAPTURE`）→ WNS +0.007 ns
板级闭环	2026-10-25 至 10-27	确认板卡型号（领航者 ZYNQ7020）；首次上板 SW 版 KAT 6/6 通过；三份镜像（SW/ISE/HW）全部板级通过并冻结哈希；IO 时序门禁 7/7 通过
信任根	2026-10-25 至 10-27	256 路 RO-PUF、DNA 与 P12 KDF 子系统落地；pblock 隔离；板上 enroll 128/128、reconstruct 一致
PS/PL 与展示层	2026-10-28	PYNQ overlay 自动构建（BD、100 MHz FCLK0、双 AXI-Lite 窗口）；板上加载与协议握手通过；三模式 289/289 KAT 与 289/289 错误 Tag 拒绝；WNS +0.034 ns、DRC 0 Error

演进过程有两处判断修正需予记录。其一，初期判断时序瓶颈位于 Cache 子系统，策略扫掠结果否证此判断：33 组策略共回收 0.824 ns（87%），剩余 0.118 ns 由三处结构性问题解决（复位扇出、乘法器组合深度、miss 通路多拍）。其二，“为异步 IO 设定宽松 input/output delay”的直觉被实测否证（见 §3.2（第三部分）），最终形态为以 `false_path` 表达语义、以形式 delay 闭合覆盖计数。

6.2 设计准则与证据分级

本作品在工程方法上遵循四项原则，后文章节结构即由此派生：

1. **契约先行**：任一加速通路先确定接口契约（编码、语义、延迟、停顿与冲刷、取消行为），再实施 RTL 与软件；禁止 RTL 与软件各自推定指令编码。
2. **实测优先于仿真，仿真优先于估算**：正文所有数值标注口径（板上实测、RTL 仿真、STA、估算），三类数据不参与同一计算。

   “板上可运行”不等于”时序已签核”，“芯片内部 WNS 为正”不等于”板级 IO 时序已签核”。
3. **可复现优先于表观完整**：每项结论须可追溯至报告文件或串口原始字节，并给出复算命令。

本文使用的四类证据标记如下，后文不再重复说明：

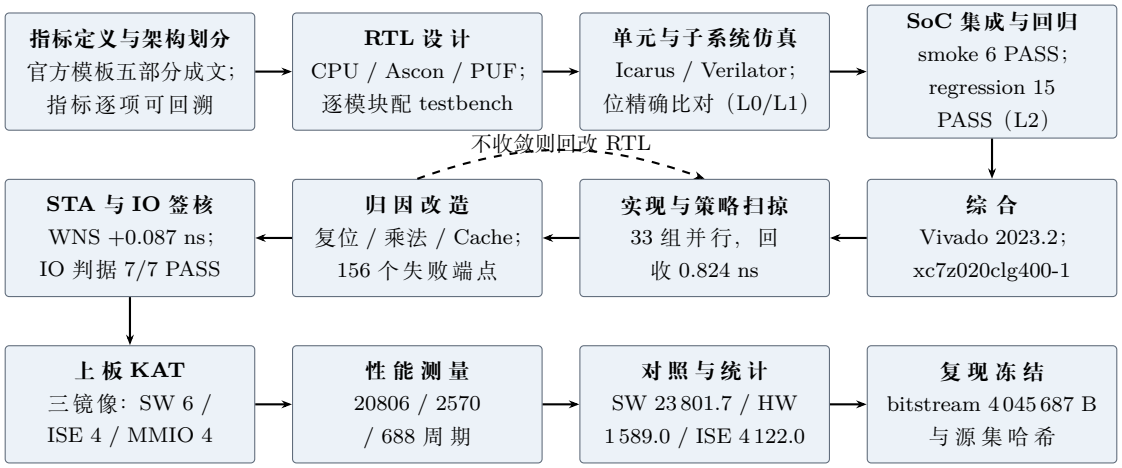


图 1-1　设计与验证流程

六个阶段及其产物。每阶段右侧标注该阶段的判据或关键读数，末段为复现冻结。

标记	含义	举例
板上实测	真实板卡运行，保留串口原始日志	20806 / 2570 / 688 周期
仿真实测	Icarus/XSim 运行 RTL 所得	17353 / 2496 / 670 周期
STA 与实现	由 Vivado 报告解析，不手工抄录	WNS +0.087、LUT 12426
建模与估算	依器件手册或算术推导，注明假设	UART IO 20 ns 上界、80 MHz 余量算术值

6.3　术语与统计口径

本节定义全文使用的关键术语与统计口径，后文不再重复解释。

术语	定义
主操作码与功能码	RISC-V 指令字中的 7 位 opcode 与 3 位 funct3。本作品的 Ascon 指令占用 RISC-V 预留的 custom-0 编码（`0b0001011`），未新造主操作码值，4 条指令以 funct3 区分
定制指令	在主操作码与功能码空间内定义、由处理器直接执行而非由库函数模拟的指令。本作品新增 4 条：`ascon.w`、`ascon.rd`、`ascon.p12`、`ascon.p8`
三种实现方案	针对同一算法的三种实现：纯软件（SW）、存储器映射加速器（MMIO，亦称 HW）、定制指令扩展（ISE）
镜像	一份 bitstream 与其 BRAM 初值文件（COE，内含固件）的组合。”三份镜像” 指 SW、ISE、MMIO 三个固件版本对应的三份 bitstream
周期加速比	同一板卡、同一工作频率、同一固件框架下，纯软件实现的 `mcycle` 增量与目标实现 `mcycle` 增量之比，$S = N_{\mathrm{SW}}/N_X$；仅反映周期数，不含频率缩放与并行度折算
端到端周期	自参数写入加速器（或固件入口）起至结果读回完成止的 `mcycle` 增量，含数据搬运与状态轮询

术语	定义
核本体周期	仅密码运算本体的周期数，不含寄存器搬运、轮询与服务层开销
板上实测 / 仿真实测 / STA	分别指真实板卡上以 `mcycle` 或串口日志取得的数值、RTL 仿真取得的数值、由 Vivado 报告解析取得的数值；三者不参与同一计算
板级 / 片上	”板级”指涉及 PCB 引脚与外部器件的层面（IO 时序、板卡连接）；”片上”指 FPGA 内部逻辑与互连
P0 阶段	指 Ascon 三种实现方案完成板级闭环的阶段；”P0 源集”为其对应 RTL 源集，”P0 三镜像”为该阶段的三份 bitstream（SW、ISE、MMIO），与后续集成 PUF 的源集不同

第二部分系统组成及功能说明

1 整体介绍

处理器采用经典五级顺序流水线结构：取指（IF）、译码与读寄存器（ID）、执行（EX）、访存（MEM）、写回（WB），并配置 I-cache、D-cache、分支预测器、旁路网络与集中式冒险控制。顶层 `cpu_core` 之下

单元	文件	职责
取指	`if_stage.v`、`if_id.v`	PC 生成、I-cache 查询、预测目标选择、冲刷响应
译码	`id_stage.v`、`id_ex.v`	指令字段解析、立即数生成、结果类型（ResultKind）判定、寄存器索引下发
执行	`ex_stage.v`、`mult.v`、`divider.v`	ALU 与移位、分支条件判定、多周期乘除、Ascon ISE 接入
访存	`mem_stage.v`、`mem_wb.v`	load/store 对齐与 D-cache 事务、总线请求
写回	`mem_wb.v`、`regfile.v`	结果选择、GPR 写端口、CSR 写端口
控制	`ctrl.v`（`stall_ctrl`）、`bypass.v`	5 位 `stall/flush` 向量、两级旁路选择
存储	`icache.v`、`dcache.v`、`bus_interconnect.v`	各 64 组 × 4 字/行；地址译码与总线状态机
系统	`csr_file.v`、`clint.v`、`uart_ctrl.v`、`gpio.v`、`cpi_cnt.v`	特权 CSR、定时器中断、串口、GPIO、CPI 事件计数
互连	`cpu_axi_bridge.v`、`axi_full_master.v`、`soc_top.v`、`top.v`、`top_pl_bram.v`	内部总线至 AXI4 master、BRAM 与外设；板级顶层与时钟复位

数据通路的关键接口约定如下：

- **指令与数据的地址空间由 Cache 区分**：I-cache 与 D-cache 独立查询同一片 BRAM 区域（统一编址，> `0x1000_0000` 起 128 KiB）；不可 Cache 的访问（MMIO）直接经总线完成，不进入 Cache；
- **结果类型于 ID 级判定**：指令在译码阶段即标注结果来源（算术、逻辑、移位、load、跳转链接、乘法），EX 级据此选择数据通路，避免在执行级重复译码；
- **旁路于 ID 级选路、于 EX 入口生效**：优先级为 `EX/MEM > MEM/WB`（`bypass.v` 中两级 MUX）；load-use 冒险以停顿方式解决，不采用推测加重放；
- **CSR 读写置于 EX 级**：`mepc`、`mcause`、`mtvec` 与陷入入口位于同一路径，使 `ecall` 与

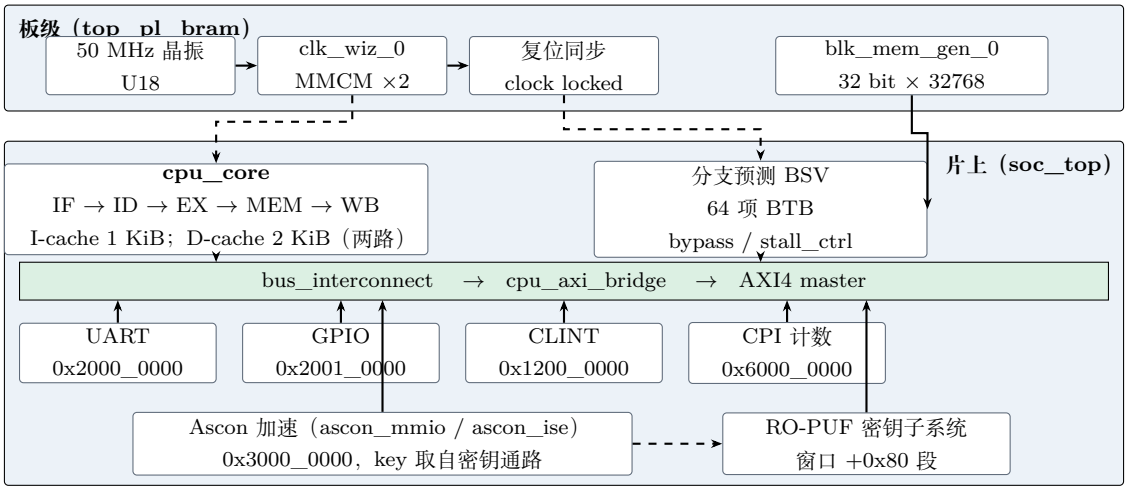


图 2-1　处理器与 SoC 总体框图

CPU 五级流水、I/D-cache、旁路与冒险控制、总线互连、AXI BRAM、UART/GPIO/CLINT/CPI，以及挂接于 MMIO 窗口的 Ascon 加速器与密钥子系统。转 LaTeX 时以 TikZ 分层绘制（流水线层、Cache 层、总线层、外设层）。

`mret` 的 PC 修复仅有单一实现。

2　硬件系统介绍

2.1　硬件整体介绍

工程入口为 `board/vivado_zynq7000_xc7z020_ascon/riscv_vivado_project/riscv_vivado_project.xpr`，part 为 `xc7z020clg400-1`，顶层为 `top_pl_bram`。

P0 三份镜像对应同一源集，工程内共 43 个文件：`src/rtl/*.v` 31 个、`src/rtl/ascon/*.v` **8 个**（该目录共 9 个 `.v`，其中 `ascon_aead_2_tb.v` 为 testbench，未纳入工程）、`const.xdc` 1 个、XCI 3 个（31+8+1+3 = 43，与工程自检 `CHECK NFILES=43` 一致；COE 为 BRAM 初值文件，另计）。PUF 阶段另增 `keys/*.v` 5 个与 pblock XDC 1 个。

类别	内容	说明
RTL	`src/rtl/*.v`（31 个，含 `defines.v`）	CPU、Cache、总线、外设、`top` 与 `top_pl_bram`
密码 RTL	`src/rtl/ascon/*.v`（9 个，含 1 个 TB）	`ascon_permutation`、`ascon_round`、`ascon_word`、`ascon_aead_core`、`ascon_aead_2`、`ascon_ise`、`ascon_mmio`、`crypto_aead`
密钥 RTL	`src/rtl/keys/*.v`（5 个）	`device_dna`、`ro_unit`、`ro_puf_core`、`puf_key`、`key_mmio`（PUF 阶段）
约束	`const.xdc`	1 个 `create_clock`（U18，20 ns）与 8 条 `PACKAGE_PIN` 约束（`clk` 加 7 个异步 IO），另有 `reset_btn` 的 false_path

类别	内容	说明
约束（PUF）	`build/keys_ro_pblock.xdc`	RO 阵列 pblock `X60Y115:X113Y149` 与 `DONT_TOUCH`
IP	`clk_wiz_0`（6.0 rev17）、`axi_bram_ctrl_0`（4.1 rev13）、`blk_mem_gen_0`（8.4 rev12）	以 XCI 纳入版本库，不提交生成缓存
初值	`ascon_sw.coe`、`ascon_ise.coe`、`ascon_hw.coe`、`ascon_keys.coe`	由脚本自 `.elf` 生成，`Load_Init_File=true`

时钟与复位：板载 50 MHz 晶振经 `clk_wiz_0` MMCM 倍频至 100 MHz 得 `clk_100m`；复位为低有效按键，经 `clock locked` 与同步链释放（`top_pl_bram.v`）。`RESET_TYPE=ACTIVE_HIGH` 与顶层的 `.reset(~rst_n_raw)` 相匹配，该极性关系在恢复 XCI 时经专门核对；极性相反将表现为” 上电偶发不启动”，属板级最难定位的故障类型之一。

更换镜像的完整流程如下，其中 COE 的更换须在 GUI 对话框中完成，直接修改文件不被 Vivado 识别：重新定制 `blk_mem_gen_0`（Re-customize IP）并将 Coe File 指向目标 COE；Generate Output Products （Out-of-context per IP）；校验 `.mif` 指纹；Run Synthesis（默认策略）；Run Implementation （`Performance_ExploreWithRemap`）；Generate Bitstream；记录 bitstream 的 SHA-256；经 JTAG 下载并采集串口日志。镜像指纹的判据为 `.mif` 第 3 与第 4 个字：SW 版第 3 字为 ···`A197`；ISE 与 HW 版第 3 字为 ···`2197`，第 4 字分别为 `1E01_8193` 与 `8101_8193`。以相邻两条指令的编码差异区分镜像，较依据文件名更为可靠。

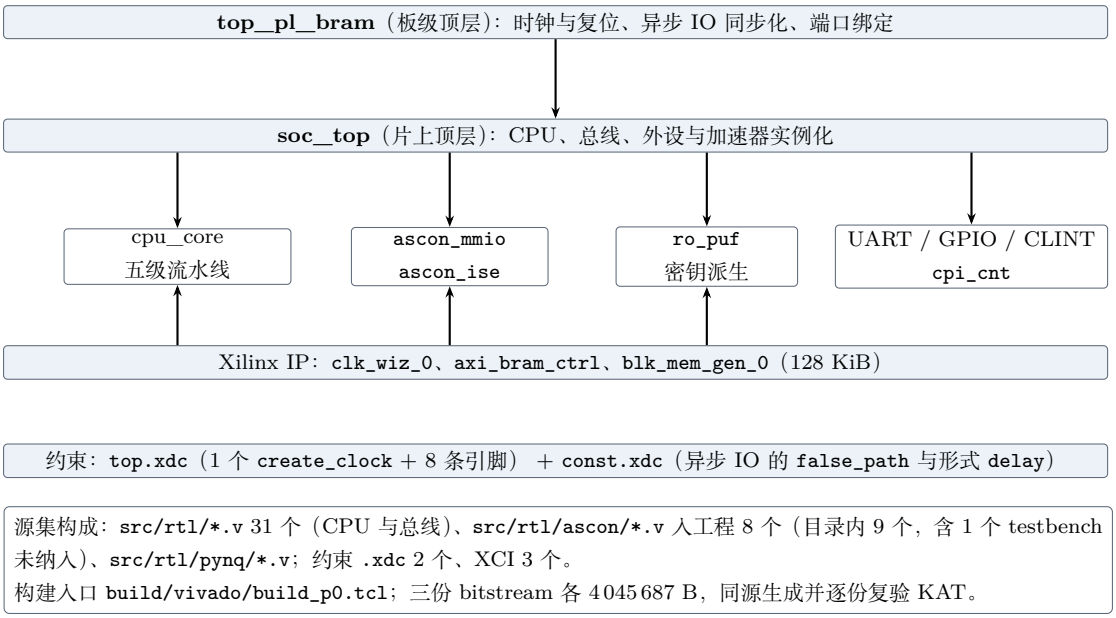


图 2-2 板级工程结构

‘top_pl_bram‘（50 MHz 输入、复位与时钟管理）→ ‘top‘（AXI4 master）→ ‘soc_top‘（CPU、Cache、总线、外设、Ascon 与密钥子系统）→ IP（‘clk_wiz_0‘、‘axi_bram_ctrl_0‘、‘blk_mem_gen_0‘）；配 Vivado 层次截图或 TikZ 框图。

2.2 机械设计介绍

本作品为纯电子系统，不含机械结构与运动部件。板卡采用正点原子领航者 ZYNQ7020 开发板，以底板 + 核心板的两段式结构装配；设计成果全部位于 PL 内部逻辑与 PS 端软件，不需要额外的机械加工件、传感器支架或外部供电模块。板级连接的仅有 50 MHz 时钟源、USB 串口（UART）、两个用户按键、两个用户 LED 与以太网口，其中串口与网口分别用于固件日志输出与 PYNQ 远程访问。

2.3 电路各模块介绍

本节按”处理器核 → 密码加速器 → 密钥子系统 → PS/PL 接口”的顺序介绍各电路模块。处理器核部分（§2.3.1 至 §2.3.9）说明五级流水线的取指、译码、执行、访存与写回，以及 Cache、分支预测、总线与异常处理；加速器部分（§2.3.10 至 §2.3.14）说明 Ascon-AEAD128 的标准参数、三种实现方案与可比性边界；密钥子系统部分（§2.3.15 至 §2.3.19）说明 RO-PUF 的安全模型、结构、综合问题与板上结果；接口部分（§2.3.21 至 §2.3.25）说明 PS/PL 数据通道的方案论证、寄存器定义、服务协议与 overlay 构建。

各模块的描述统一给出”职责、接口、关键时序、验证证据”四项要素，，凡涉及 RTL 行为的表述均以源码为准（行号见附录 C 证据索引），不以设计意图代替实现现状。

2.3.1 取指单元（IF） 取指级的职责为每周期产生一个 PC 及一条指令，需同时处理 Cache 命中、分支预测与流水线冲刷三项相互耦合的条件。

- **PC 来源优先级**（由高至低）：陷入与中断入口、EX 级分支重定向（`jump_en`）、分支预测目标、`pc + 4`；
- **I-cache 查询**：容量 1 KiB，64 组 × 4 字/行，直接映射；tag 命中时当拍返回。未命中时进入 `FILL` 状态并向总线发出一次 burst 请求，`ctrl.v` 收到 `icache_stall` 后冻结 IF 与 IF/ID；
- **预测器接口**：取指级以当前 PC 组合读出 `pred_taken` 与 `pred_target`（分布式 RAM，零延迟），预测命中时当拍改选 PC，不增加流水级；
- **冲刷**：`flush[4]` 清除 IF，`flush[3]` 清除 IF/ID。分支重定向（`jump_en`）时 `ctrl.v` 置 `flush[3:2]`（清除 IF/ID 与 ID/EX 两级，作废两条较年轻指令）；陷入、`mret` 与外部中断时置 `flush[3:1]`（额外清除 EX/MEM）。各级冲刷的流水线代价随清除级数而变，本文不作未经实测的拍数结论。

2.3.2 译码单元（ID） 译码级完成字段解析与指令意图标注，为流水线中控制逻辑最密集的一级。

- **立即数生成**：I、S、B、U、J 五种格式并行生成，由 opcode 选择；
- **寄存器堆**：32 × 32-bit，双读一写；`x0` 恒为零（写端口屏蔽）；写口位于 WB 级，同拍读写同一地址返回旧值，跨级正确性由旁路保证；
- **结果类型（ResultKind）**：将指令映射为六类结果来源（算术、逻辑、移位、load、跳转链接、乘法），该标注随流水线传递至 EX/MEM/WB，为后级”不再重复译码”的基础；

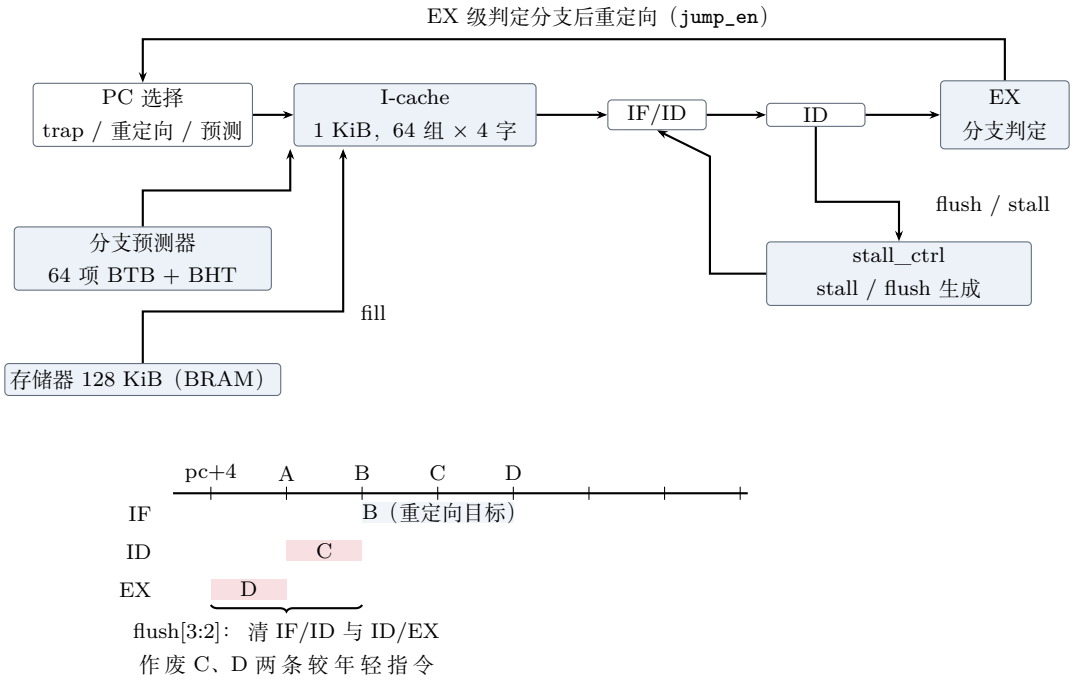


图 2-3 IF 级数据通路与 miss/冲刷时序

PC 选择、I-cache 查询、预测改选、miss FILL 与分支冲刷（‘flush[3:2]‘ 两级）；波形示意按 RTL 的 ‘flush‘ 位置标注。

- **CSR 指令与陷入**：`csrrw`、`csrrs`、`csrrc`、`ecall` 与 `mret` 于 ID 级识别为 SYSTEM 类，具体读写与 `mepc`、`mcause` 更新在 EX 级完成；
- **load-use 检测**：ID 级比较 `id_ex.rd` 与当前 `rs1`、`rs2`，命中时置 `stall_ld`，由 `ctrl.v` 冻结 IF 与 IF/ID（`stall[4:3]`），并在 ID/EX 插入气泡（`flush[2]`）；EX/MEM 不再清除。

2.3.3 执行单元（EX） 执行级为多功能并行与多周期串行相结合的混合结构，本作品的两处关键改造均作用于该级。

并行功能单元：ALU（加、减、逻辑、比较、移位）、分支条件判定、跳转链接地址生成与 CSR 读写均在单周期内完成；乘法器 `mult.v` 完成 32 × 32 有符号与无符号乘法（RV32M 的 `mul`、`mulh`、`mulhsu`、`mulhu`），综合后被映射为 4 个 DSP48E1（实现报告 DSP 利用率 4 / 220），映射结果由工具的乘法推断产生而非显式例化 DSP 原语；除法器 `divider.v` 为迭代式多周期实现（`div`、`divu`、`rem`、`remu`）。

多周期与握手：执行级向 `ctrl.v` 发出 `ex_stall_req`，后者冻结 IF、IF/ID 与 ID/EX 并在 EX/MEM 插入气泡。多周期单元完成时将结果与 `done` 一并锁存，直至被确认（ack）方予释放。乘法器于本轮改造中增加 `mul_ack` 与 `cancel` 两个内部端口，其契约为：输入寄存器至乘积寄存器，接受后下一拍乘积与 `mul_done` 有效；`done` 与结果保持至 ack；cancel 或 reset 取消且不产生完成信号。该契约使”乘法结果在 EX 级保持”与”陷入取消”两项行为互不干扰。

Ascon ISE 接入点：custom-0（opcode `0x0b`，funct7 = 0，funct3 区分四类操作）在该级

被识别为一类带状态的多周期指令，其读写对象为 ISE 内部的 10 个 32-bit 状态字（§2.3.13），而非通用寄存器。执行级以 hold 方式使流水线跨越 P12/P8 的多拍执行，`done` 保持至指令推进；“完成有效方予退休” 的判据保证该指令不会因跨过完成的停顿而被重复执行。

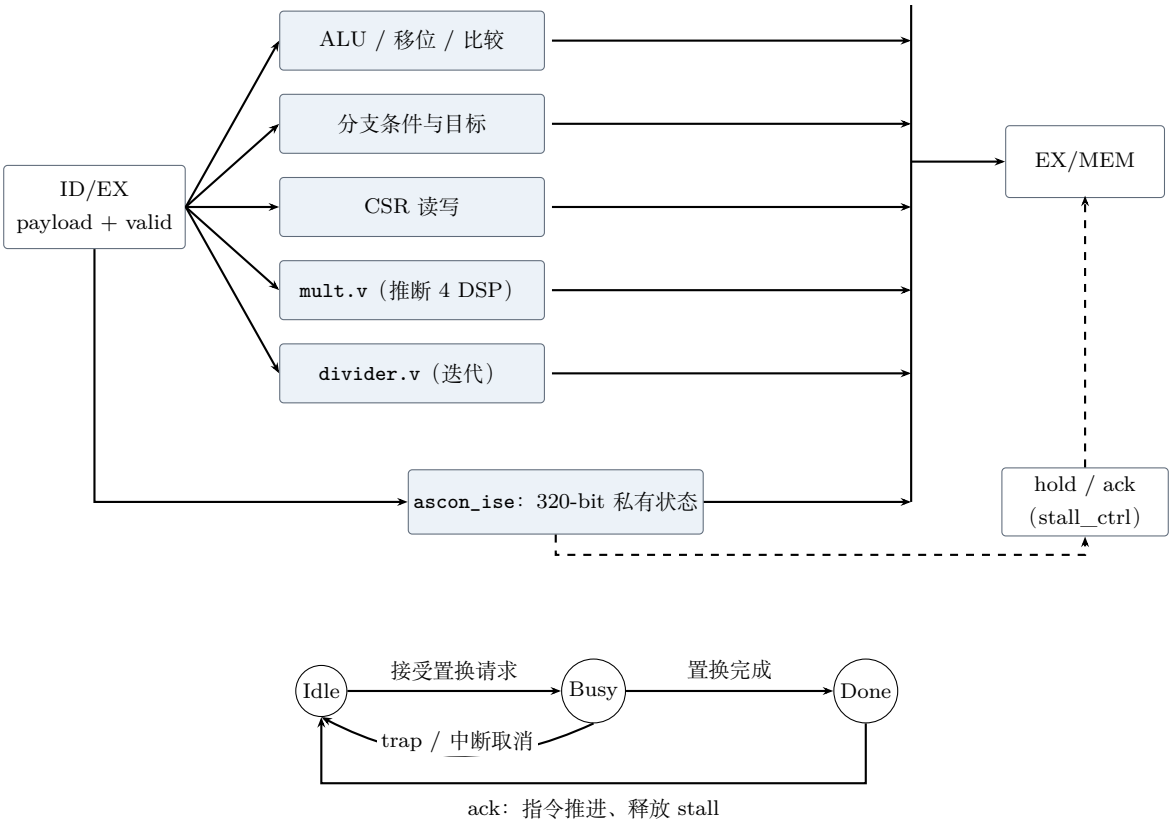


图 2-4　EX 级功能单元与多周期握手状态图

含 ‘Idle → Busy → Done → ack 转换、陷入与中断取消分支，以及 ISE hold 与 ALU 单拍路径的复用关系。

2.3.4 访存与写回（MEM/WB）及 Cache 结构　访存级将执行级的地址与数据转换为 D-cache 事务或总线事务；写回级完成 GPR 与 CSR 写回。

D-cache 结构（`dcache.v`）：容量 2 KiB，64 组 × 4 字/行，两路组相联，采用 write-through 策略（store 不等待填充），tag 与数据分存；不可 Cache 的地址（MMIO 窗口）绕过 Cache，改为单拍注册请求。状态机为 `IDLE / MISS_CAPTURE / FILL` 三态：

- `IDLE`：查询 tag，命中即返回；
- `MISS_CAPTURE`：本轮新增状态，将 miss 的 tag、index、way 及请求属性锁存一拍，使”miss 判定至总线请求” 不再经过单一长组合路径；
- `FILL`：等待总线填充完成，逐字写入 way。

I-cache：容量 1 KiB，64 组 × 4 字/行，直接映射，状态机为 `IDLE/FILL` 两态，未命中时整行填充。

写回：`mem_wb` 选择访存结果、load 数据、PC + 4 或 CSR 读值，驱动 `regfile` 写口；`x0` 于写口屏蔽，“读 x0 恒为零” 由寄存器堆本身保证，写回级不再作特判。

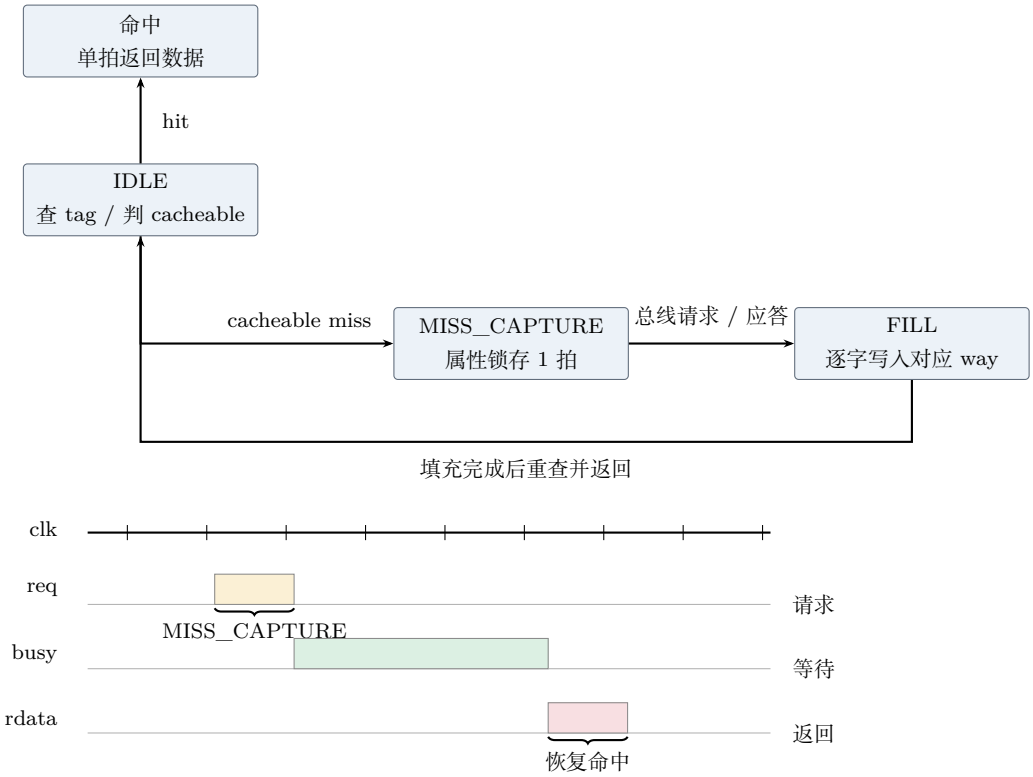


图 2-5 D-cache 与总线请求时序

命中单拍、'MISS_CAPTURE' 一拍锁存、'FILL' 逐字写入，以及 store 与 MMIO 的绕过通路。

2.3.5 CSR、异常与计数 `csr_file.v` 实现的 CSR 集合为：`mstatus`（0x300）、`misa`（0x301）、`mie`（0x304）、`mtvec`（0x305）、`mscratch`（0x340）、`mepc`（0x341）、`mcause`（0x342）、`mtval`（0x343）、`mip`（0x344）、`mvendorid`（0xF11）、`marchid`（0xF12）、`mimpid`（0xF13）、`mhartid`（0xF14），以及 64-bit `mcycle` 与 `mcycleh`（0xB00 / 0xB80）。其中 `mvendorid` / `marchid` / `mimpid` / `mhartid` 返回常量 0。

- `mcycle` 为**唯一周期来源**：每个非复位时钟加一，RV32 经 `0xb00`（低）与 `0xb80`（高）访问，软件按 high→low→high 顺序读取并校验一致性后重读。本作品全部周期加速比均以 `mcycle` 差值计算，不以事件计数之和替代——后者（`cpi_cnt` 的 inst、stall、flush 事件）存在重叠，不可相加为周期数；
- **陷入流程**：`ecall` 于 EX 级写入 `mepc`（当前 PC）与 `mcause = 11`，跳转至 `mtvec`；`mret` 自 `mepc` 恢复 PC；外部中断经 `ireq_external = gpio_irq | uart_irq | ascon_irq` 汇聚，抢占时由 `stall_ctrl` 清除较年轻指令；
- **计数与 CPI**：`cpi_cnt.v` 挂接 `0x6000_0000` 窗口，提供 `probe_inst_valid`、`probe_stall_ld`、`probe_stall_md` 与 `probe_flush_br`（每事件计权 2）四组 64-bit 计数；CPI 以同一区间的 cycle delta 与已退休指令 delta 计算，不采用事件加权和。

2.3.6 分支预测 预测器为 64 项单表结构（`branch_predictor.v`，`INDEX_WIDTH = 6`）：

- **表项**：tag 表、BTB 目标表（分布式 RAM，`ram_style="distributed"`，整体 `dont_`

touch）、2-bit 饱和计数器 BHT 与打包 valid 向量；

- **索引**：`index = pc[7:2]`（指令 4 字节对齐，低 2 位恒为零），`tag = pc[31:8]`；
- **预测**：仅当 `valid && tag_match && bht >= 2'b10` 时判为 taken，目标取 BTB 中记录的历史目标；
- **更新**：分支进入 EX 级后训练（`update_en`、`pc_ex`、`actual_taken`、`actual_target`），并按 2-bit 状态机更新 BHT（`00↔01↔10↔11`，初值为弱不跳）。误预测冲刷由 `ctrl.v` 统一处理，预测器不参与冲刷。

采用”小型表 + 分布式 RAM”而非大容量 BTB，系针对本平台的容量与时序折衷：在 100 MHz、LUT 资源受限的 7020 器件上，该结构的面积与时序代价均较低；就本 SoC 的主要负载（AEAD 分块循环与基准测试主循环）而言，其命中率已属充分。性能增益的主要来源为密码算法本体的周期加速比（§6），而非预测器容量。

2.3.7 总线与 SoC 外设 `bus_interconnect.v` 按地址高位将读写请求译码至各从设备；`cpu_axi_bridge.v` 与 `axi_full_master.v` 将内部事务转换为 AXI4 master（ID 1 bit、WSTRB = XLEN/8，支持单拍与 burst）。

窗口	基址	从设备	说明
MEM	`0x1000_0000`	AXI BRAM Controller → Block RAM	128 KiB，代码与数据统一空间，可 Cache
CLINT	`0x1200_0000`	`clint.v`	`msip`、`mtimecmp`、`mtime`，提供软件定时中断
UART	`0x2000_0000`	`uart_ctrl.v`	115200 8N1，THR/RBR/status，FIFO，读 FIFO 有副作用
GPIO	`0x2001_0000`	`gpio.v`	LED 控制、按键事件锁存（读清），两级同步与消抖
ASCON	`0x3000_0000`	`ascon_mmio.v`（密钥窗口位于 `+0x80`）	AEAD 加速器与 PUF 密钥状态回读（§2.3.12、§7）
CPI	`0x6000_0000`	`cpi_cnt.v`	四组 64-bit 事件计数

MMIO 事务不进入 Cache，亦不参与 Cache 一致性维护；`ascon_rdata = ascon_rdata_core | key_rdata` 以”未选中恒为零、按位或合并”的方式将两个子窗口合至单条读数据通路，其优点在于无需修改共享的 `bus_interconnect.v`，符合本作品”最小化公共路径改动”的约束。

2.3.8 流水线控制与冒险处理 `ctrl.v` 输出两个 5 位向量 `stall[4:0]` 与 `flush[4:0]`，位序为 `{IF, IF/ID, ID/EX, EX/MEM, MEM/WB}`，优先级由高至低如下表。

顺序	事件	`ctrl.v` 中的动作
1	`inter_req`（外部中断）或 `jump_trap_en`（陷入，同一条件分支）	`flush[3:1]`；`jump_sel = 10`（中断 / 陷入向量）

顺序	事件	ctrl.v 中的动作
2	jump_trap_out（mret）	flush[3:1]；jump_sel = 11（mepc）
3	dcache_stall	stall[4:0] 全冻结
4	dmem_axi_stall_req（store 等待总线）	stall[4:0] 全冻结
5	ex_stall_req（多周期单元与 ISE）	stall[4:2]；flush[1]（EX/MEM 插气泡）
6	jump_en（分支重定向）	flush[3:2]（IF/ID 与 ID/EX）；jump_sel = 01
7	load_use_rs1 / load_use_rs2	flush[2]（ID/EX 插气泡）；stall[4:3]（IF 与 IF/ID）
8	icache_stall / fence_i_stall	stall[4:3]；flush[2]

表中次序为 ctrl.v 内 if-else 链的实际判定顺序，据此，Cache 与总线停顿优先于分支重定向，中断与陷入同属首个条件分支。

该层为加速指令正确执行的关键：ISE 的多拍执行属第 5 项，陷入与中断属第 1 项；两者叠加时以 “陷入优先、ISE 状态一并清零、不发布迟到结果” 为语义（§2.3.13）。

2.3.9 与加速及时序收敛相关的结构改造

序号	改造内容	位置	动机
1	器件与工程迁移：Kintex-7 → Zynq-7020，重建 3 个 IP 与 1 个 XDC，PLL 由 50 MHz 倍频至 100 MHz	board/vivado_zynq7000_xc7z020_ascon/	早期工程的目标器件为 xc7k325t，与提交板卡不符
2	D-cache 增加 MISS_CAPTURE 状态，miss 属性锁存一拍后再发总线	dcache.v	消除”miss 判定至总线请求” 的长组合路径（11 个失败端点）
3	store 与 MMIO 绕过 MISS_CAPTURE，恢复单拍注册请求	dcache.v	MISS_CAPTURE 对不可 Cache 访问仅增加延迟，据此回收周期代价
4	乘法器输入与输出寄存器化，并引入 mul_ack、cancel 握手	mult.v、ex_stage.v	乘法器组合深度对应 48 个失败端点、−0.105 ns
5	ISE 状态寄存器改为局部复位控件	ascon/ascon_ise.v	全局复位扇出对应 93 个失败端点、−0.128 ns
6	新增 Ascon MMIO 窗口、ISE 指令与密钥窗口	ascon/、keys/、soc_top.v	本作品的核心功能增量
7	UART 起始位检测审计（d0 与 d1/d2）	uart_rx.v	结构探针表明 start_en 使用了未经消抖的首级信号（§3.3.6，[待确认是否修改 RTL]）

前五项与加速功能无直接关联，但决定该 SoC 能否在 100 MHz 收敛。第 4 章将说明：33

组策略扫掠将 WNS 自 -0.942 ns 提升至 -0.118 ns，而剩余 0.118 ns 须由上述结构改造回收。

2.3.10 算法标准与参数定义 Ascon 的安全核心为 320 bit 状态 $S = (s_0, s_1, s_2, s_3, s_4)$，每个分量 64 bit，认证加密通过” 吸收与挤出 + 置换” 实现。AEAD128 的参数如下表。

参数	取值	本作品的处理
key	16 byte / 128 bit	K_0 取低 64 bit，K_1 取高 64 bit
nonce	16 byte / 128 bit	N_0、N_1 同理
tag	16 byte / 128 bit	完整输出，不截断
rate / capacity	128 bit / 192 bit	每块吸收 16 byte
初始向量	整数 `0x0000_1000_808c_0001`，字节串 `01 00 8c 80 00 10 00 00`	常量以位移式匹配
轮常量	`f0 e1 d2 c3 b4 a5 96 87 78 69 5a 4b`（第 0 至 11 轮）	P12 取前 12 个，P8 取后 8 个
域分离	$s_4 \oplus= 2^{63}$（`0x8000_0000_0000_0000`）	于 AD 与 PT 之间插入

轮函数。第 i 轮先执行常量加与 S-box，再执行线性扩散：

$$
\begin{aligned}
&s_2 \oplus= c_i \\
&s_0 \oplus= s_4, \quad s_4 \oplus= s_3, \quad s_2 \oplus= s_1 \\
&t_0 = \overline{s_0}\, s_1, \;\; t_1 = \overline{s_1}\, s_2, \;\; t_2 = \overline{s_2}\, s_3, \;\; t_3 = \overline{s_3}\, s_4, \;\; t_4 = \overline{s_4}\, s_0 \\
&s_0 \oplus= t_1, \quad s_1 \oplus= t_2, \quad s_2 \oplus= t_3, \quad s_3 \oplus= t_4, \quad s_4 \oplus= t_0 \\
&s_1 \oplus= s_0, \quad s_0 \oplus= s_4, \quad s_3 \oplus= s_2, \quad s_2 = \overline{s_2}
\end{aligned}
$$

线性层（rotr 表示 64 bit 循环右移）：

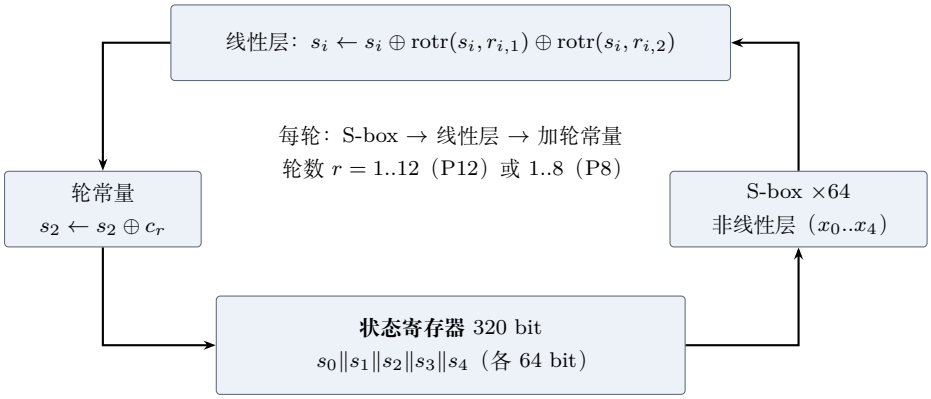
$$
s_i \oplus= \mathrm{rotr}(s_i,\, r_{i,1}) \oplus \mathrm{rotr}(s_i,\, r_{i,2}), \qquad (r_{i,1}, r_{i,2}) \in \{(19,28), (61,39), (1,6), (10,17), (7,41)\}
$$

AEAD 流程（加密方向）：

1. 初始化：$S \leftarrow IV \,\|\, K_0 \,\|\, K_1$，$S \leftarrow P^{12}(S)$，$s_3 \oplus= K_0$，$s_4 \oplus= K_1$；
2. 关联数据：按 16 byte 分块吸收并执行 P^8；非空整块之后仍须追加一个空 padding 块并再次执行 P^8；
3. 域分离：$s_4 \oplus= 2^{63}$；
4. 明文：按 16 byte 分块挤出 $C_i = P_i \oplus (s_0 \| s_1)$ 并执行 P^8；末块不足 16 byte 时先补 `0x01` 再补 `0x00`，末块不再置换；
5. 终结：$s_2 \oplus= K_0$，$s_3 \oplus= K_1$，$S \leftarrow P^{12}(S)$，$s_3 \oplus= K_0$，$s_4 \oplus= K_1$；
6. 标签：$T = \mathrm{LE64}(s_3) \,\|\, \mathrm{LE64}(s_4)$；密文与明文等长。

三种方案共用的实现要点如下，均经 1,089 组 KAT 逐字节验证，亦为最易出错之处：

- padding 为" 追加字节 `0x01` 后补 `0x00`"，在 64-bit 字中表示为 $1 \ll (8n)$，$n < 8$；
- AD 为空时既不吸收亦不 padding，但仍执行域分离；
- 整块 AD 与整块 PT 之后均须再经一个空 padding 块，不得以" 满块即结束" 处理；
- 长度须显式给出，不得由" 最高非零 bit" 反推（否则 `00` 与 `a500` 将判定错误）。



旋转常数：
Σ_0：19、28　Σ_3：10、7
Σ_1：61、39　Σ_4：7、41
Σ_2：1、6　另加 12 组 64 bit 轮常量 c_r

参数：IV = `0x0000_1000_808c_0001`；
rate = 128 bit，capacity = 192 bit；
域分离常数 2^{63}：初始化 high/low、终结与 Tag 分离；
硬件：`ascon_permutation.v`，P12 与 P8 共用同一数据通路，MMIO 与 ISE 两种方案复用同一置换核。

图 2-6　Ascon 置换数据通路

5 × 64-bit 状态、S-box 的五个与非项组合、线性层的五组双旋转与轮常量注入；标注 P12/P8 轮数差异及 ISE 内 320-bit 状态寄存器位置。

2.3.11 方案一：纯软件实现（SW）　软件实现为正确性基准，同时作为周期加速比的参照基准（分母）。生产代码位于 `ascon/ascon_c/aead/`，锁定且不可修改的参考实现位于 `sim/ascon/reference/ascon_c/`（ascon-c 固定 commit `446347f2`）。

API 契约（C）：

- `crypto_aead_encrypt(c, clen, m, mlen, ad, adlen, npub, k)`：`clen` 含 16 byte tag；
- `crypto_aead_decrypt(m, mlen, c, clen, ad, adlen, tag, k)`：成功返回 0，且 `*mlen = clen - 16`；认证失败返回 −1，`*mlen = 0`，并清除 `m[0, clen-16)` 范围内的候选明文；`clen < 16` 时返回 −1

"认证失败须清除候选明文" 为项目验收要求，非 SUPERCOP 的强制项：原始 C 参考在 bad tag 时返回 −1，但缓冲区仍保留明文，且 `mlen` 等于明文长度。本作品于调用层实施擦除与长度归零，并将" 原始 C 的真实 FAIL 行为" 作为保留证据，不予掩盖。

RISC-V 目标构建：以 xpack `riscv-none-elf-gcc 15.2.0`、`-O2` 交叉编译为裸机固件，链接至 `0x1000_0000`，经 UART 输出测试结果。周期加速比的分母即该固件在板上的 `mcycle` 增量。在 16 字节明文、warm 条件（指令已在 I-cache）下，板上实测为 **20806 周期**。

2.3.12 方案二：存储器映射加速器（MMIO / HW） MMIO 实现将整个 AEAD 实现为挂接于 0x3000_0000 的从设备，CPU 仅承担参数搬运、状态轮询与结果读取。接口契约（ascon_mmio.v 头注释，字索引 = cpu_addr[7:2]）如下表。

偏移	名称	读写	位定义与语义
0x00	CTRL	W	[0] START（自清脉冲，仅空闲态接受）；[1] ENCRYPT（1 加密，0 解密）；[2] ZEROIZE（仅空闲态接受，同一写中优先于 START）
0x04	STATUS	R	[0] BUSY；[1] DONE；[2] FAIL（解密 tag 校验失败）
0x08	ADLEN	W	AD 字节数
0x0C	MLEN	W	消息字节数
0x10--0x1C	KEY	W	K_0 至 K_3（小端序字；硬件密钥模式下被忽略）
0x20--0x2C	NONCE	W	N_0 至 N_3
0x30--0x3C	TAGIN	W	解密时待校验的 tag
0x40	AD_WDATA	W	推入一个 AD 字（内部指针自增）
0x44	MSG_WDATA	W	推入一个消息字（加密为明文，解密为密文）
0x48	OUT_RDATA	R	弹出一个输出字（加密为密文，解密为明文），指针自增
0x4C--0x58	TAGOUT	R	加密计算所得 tag
0x80--0xD0	密钥窗口	R	PUF 密钥子系统的 STATUS、CTRL、DNA、QUALITY、REF、MASK、DITHER 与 RAW（§7）

以下四项行为构成”可用外设”而非”可计算模块”的判据，须写入契约：

1. **BUSY 期间全部写操作被忽略**，包括 ZEROIZE、START、key/nonce、长度与输入缓冲；
2. **输出读门禁**：OUT_RDATA 仅在 DONE && !BUSY && !FAIL 且未超出有效字数时返回数据并推进指针，其余情形返回零且不推进。认证失败不得发布明文由硬件保证，不依赖软件自觉；
3. **ZEROIZE**：清除 key、nonce、tagin、长度与状态，以及全部 AD、MSG、OUT 数组，并经子核心复位清除适配器缓冲、$s_0..s_4$、候选明文与 P8/P12 私有副本。DONE 本身不等于清理，取完结果后须显式清零；
4. **长度上限**：硬件实际支持 AD 与消息各 0 至 16 byte（参数名 MAX_AD_BYTES=256 仅为缓冲上限，不代表多块能力），超长请求显式返回 FAIL。该限制为 P0 的边界条件：板级 MMIO 周期加速比仅对 ≤16 byte 成立，正文不将其泛化为”任意长度”。

延迟结构上，MMIO 端到端周期由参数搬运（key、nonce、AD 与消息的逐字写入）、轮询往返与密码核本体三部分组成。板上实测 16 字节加密为 **688 周期**，其中核本体 **103 周期**，其余约 85% 为寄存器搬运与轮询开销。该比例属外设式加速的结构性成本，亦为 ISE 路线存在之依据。

2.3.13 方案三：定制指令扩展（ISE） ISE 将置换并入 CPU：320 bit Ascon 状态作为执行级的专用寄存器堆，由 4 条定制指令访问。指令占用 RISC-V 预留的 custom-0 编码（0b0001011），**未新造主操作码值**，4 条指令以 funct3 区分。

funct3	助记符	操作数	周期	语义
000	ascon.w	rs1 为数据，rs2[3:0] 为索引	1	state[32*idx +: 32] = rs1
001	ascon.rd	rs1[3:0] 为索引，rd 为结果	1	rd = state[32*idx +: 32]
010	ascon.p12	—	多周期	state = P12(state)
011	ascon.p8	—	多周期	state = P8(state)

状态映射固定为 10 个 32-bit 字：s0 低与高、s1 低与高，直至 s4。非法索引（不在 0 至 9 范围）读返回零、写被忽略；该行为属显式容错约定，不产生非法指令异常。保留 opcode 与 funct 编码的全局审计仍在待办之列，正文不声称已覆盖全部指令空间。

实现细节与生命周期如下：

- perm_start 为电平信号（指令位于 EX 且未完成期间保持有效），内部状态机对其打一拍脉冲，完成后锁存结果；据此，流水线 hold 跨越完成不会导致重复执行；
- **取消**：陷入与中断经 cancel 丢弃在飞操作并清零 ISE 状态，且不发布迟到结果；普通分支冲刷仅清除较年轻指令，不影响已完成的置换；
- **局部复位寄存器化**：P12/P8 的 1280-bit 私有复位扇出原挂接于全局复位的组合锥（93 个失败端点、−0.128 ns），改为" 取消当拍清架构 state，下一拍清私有 s_reg/s_out"，接受与 p_start 同拍释放局部复位，不增加正常执行的置换周期（§3.1（第三部分））；
- **无上下文保存与恢复 ABI**：陷入取消后不可自中间轮续算，调用者须串行独占或整体重算。当前
- **软件封装**：以 asm volatile 手写 intrinsic（不改动 GCC backend），逐字读写状态并调用 P8/P12；软件侧 AEAD 循环与 MMIO 版本共用同一调用顺序与测试向量，故两版密文与 tag 可逐字节比对。

板上实测 16 字节加密为 **2570 周期**，相对软件实现的周期加速比为 8.10×。相对同源仿真值（2496 周期），板上增量为 **+3.0%**，其原因在于 ISE 与软件的访存密度相近：加速收益来自" 置换不再展开为数十条整数指令"，而非" 减少数据搬运"。

2.3.14 三种方案的可比性边界

维度	SW	MMIO（HW）	ISE
数据通路	全部于 CPU 内逐字运算	CPU → MMIO 总线 → 专用核	CPU EX 级内，状态寄存器
单次 AEAD 的 CPU 动作	纯计算	写 key/nonce/AD/msg/len → START → 轮询 STATUS → 读 OUT/TAG	写状态 → P8/P12 → 读状态
板级周期（16 B）	20806	688（端到端，核本体 103）	2570

维度	SW	MMIO（HW）	ISE
周期加速比（相对 SW）	1.0×	30.24×	8.10×
长度能力	多块（测试至 32 B）	仅 ≤16 B	多块（测试至 32 B）
主要成本	无额外硬件	寄存器与窗口逻辑、轮询开销	指令译码与 EX 级多周期、状态复位扇出
面积影响	—	加速器本体	ISE 状态与置换器（与 MMIO 共用 `ascon_permutation.v`）
时序风险	无	MMIO 完成路径（§3.1（第三部分）最差路径）	EX 级 hold 与状态复位扇出
适用场景	低频调用、零硬件成本	中等长度消息（当前硬件上限 ≤16 B）、批处理、CPU 可休眠	中小消息、无总线往返、与代码紧耦合

上述结果不宜读作”性能强弱排序”，而应理解为两条技术路线的取舍：MMIO 的周期加速比更高，但其能力受 16 byte 寄存器窗口与轮询开销约束，且 CPU 在轮询期间不可用；ISE 的周期加速比相对较低，然其状态位于流水线内，无总线往返与窗口上限，适用于”单次调用、多次置换”的嵌入式调用模式。三者置于同一 SoC、同一套向量与同一块板上测量，方能为该取舍提供实测依据。

证　据 `docs/freeze_checkpoint_20261027.md §3`、`docs/ascon_key_management.md`、`docs/ascon_standard_baseline.md §2`、`src/rtl/ascon/ascon_mmio.v`、`src/rtl/ascon/ascon_ise.v`

2.3.15 安全模型　三个角色职责互异，任一单独均不足以构成安全性：

角色	性质	作用
RO-PUF 响应	秘密，由制造偏差决定，仅该芯片可测得	提供不可复制的熵源
57-bit Device DNA	公开可读，非秘密	将密钥绑定于该芯片（更换芯片则无法重建）
Ascon P12 KDF	单向函数	混合上述两类比特，生成 128-bit 密钥

密钥派生关系为：

$$s = \text{KDF_CTX} \oplus \{\text{”KEYGEN04”},\ \text{”KEYGEN03”},\ \text{puf}[127{:}0],\ \{7'b0,\ \text{dna}[56{:}0]\}\},$$
$$K = P^{12}(s)[127{:}0]$$

PUF 位与 DNA **均参与派生**：缺任一方均无法重建同一密钥。仿真已证：helper 被破坏时系统走 DNA-only 回退，所得密钥与纯 DNA 模式完全一致，表明 PUF 位确已参与派生而非被旁路。DNA 仅 57 bit 且公开，单独使用仅构成设备绑定，不构成秘密。

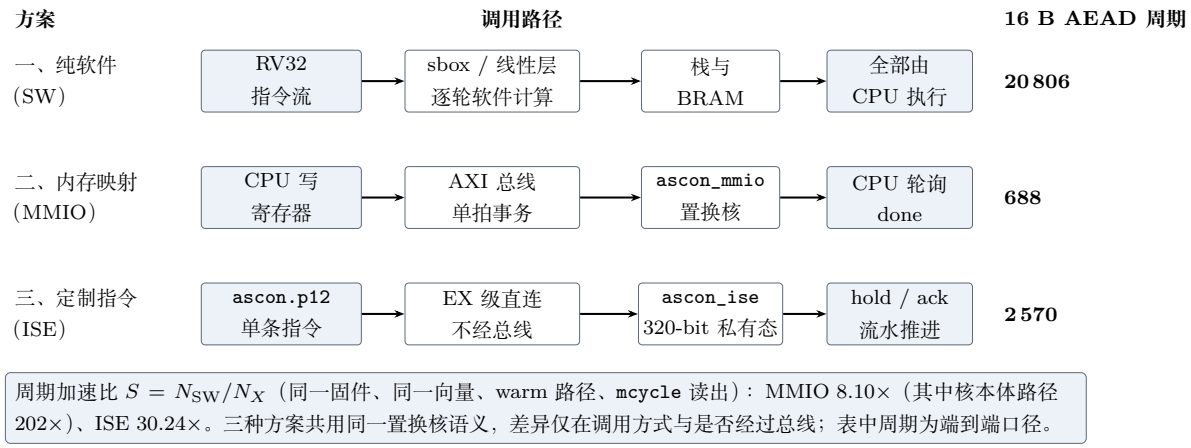


图 2-7　三种实现的调用路径对比

SW（全部于 CPU 内完成）、HW（CPU 经寄存器搬运至加速器并轮询）、ISE（CPU 直接执行 custom-0，状态位于 EX 级）；标注三者的数据搬运量与关键延迟分项（搬运、轮询、算法本体）。

2.3.16 结构

- **256 路环形振荡器**：每路为 5 级 LUT 环，逐级以 `DONT_TOUCH` 固化（否则综合将 5 级环压缩为单个 LUT，见 §2.3.17）；RO 阵列置于独立 pblock `X60Y115:X113Y149`，与 CPU 物理隔离；
- **以 RO 自身为时钟计数**：采用 `always @(posedge osc[k])` 计数，而非以系统时钟检测 RO 边沿。7 系列 LUT 环的典型频率为 150 至 300 MHz，高于 100 MHz 采样时钟，边沿检测将产生混叠，计数结果仅反映相位而非频率。该修正为本轮最重要的设计变更；
- **双时钟域**：核心域 100 MHz 承担 DNA 读取、KDF、密钥寄存器与 MMIO 握手；PUF 域由 `ro_clk = clk/4 = 25 MHz`（经 BUFG）承担 RO 使能、窗口、清零、快照、逐位比较与表决。域间仅传递两个翻转脉冲（`req_tgl`、`done_tgl`，两级同步），`raw` 与 `dither` 于 done 之后保持静态，避免 256 个计数器进入 100 MHz 关键路径；
- **多次采样与表决**：每个挑战窗口 $W = 16384$ 个 `ro_clk`，重复 $M = 5$ 次并逐位多数表决；不稳定位记为 `dither`，重建时以 `mask` 屏蔽。采样总耗时为 $M \times (\mathrm{DWELL} + W) = 5 \times (256 + 16384)$ 个 `ro_clk`，约 **3.3 ms**；
- **MMIO 窗口**：位于 Ascon 窗口 `+0x80` 段（21 个字），可读 DNA、QUALITY、REF、MASK、DITHER 与 RAW，**密钥不在该窗口内**：`key` 仅存在于 `puf_key → ascon_mmio` 的内部通路，CPU 既不可写亦不可读。

2.3.17 综合与实现中的三处关键问题

1. **5 级环被合并为单个 LUT**：初版以 `assign` 链构建环，综合后 DRC 报 `LUTLP-1 Combinatorial Loop Alert: 1 LUT cells form a combinatorial loop`，`STAGES` 参数完全失效，部分 challenge 位恒定，熵显著下降。修正方式为改用显式 LUT 原语并逐级 `DONT_TOUCH`；
2. **组合环须显式声明，且不能置于 XDC**：约束文件于综合前读入，此时 net 尚不存在，`get_nets` 返回空集，属性被静默丢弃。正确做法为挂接至 `STEPS.OPT_DESIGN.TCL.PRE` 的

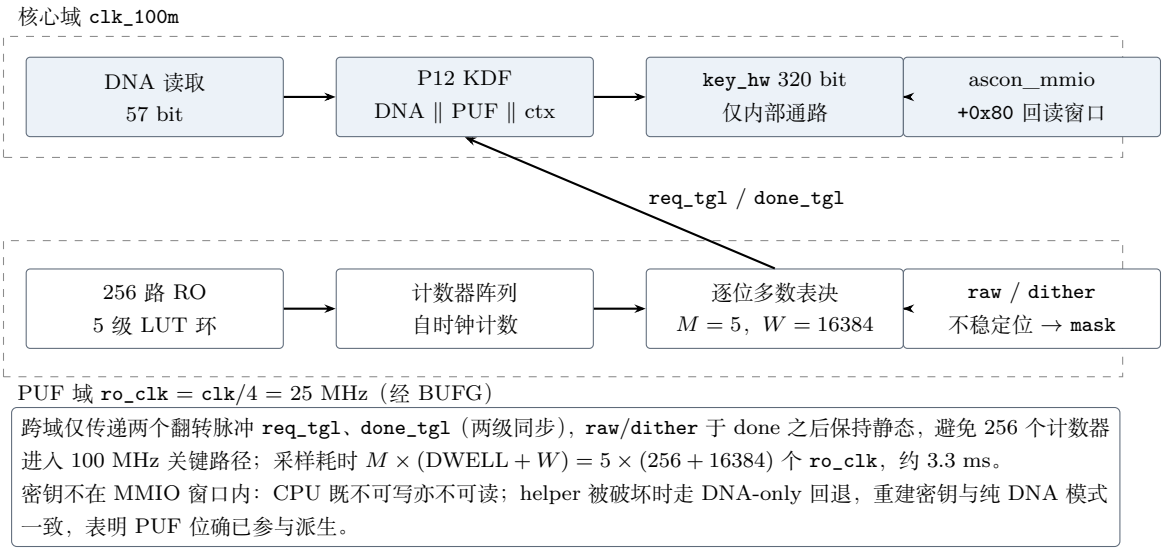


图 2-8 PUF 密钥子系统结构

256 路 RO 阵列与 pblock、以 RO 自时钟的计数器、双时钟域握手、多数表决与 dither、P12 KDF，以及 'ascon_mmio' 的 'key_hw' 通路与 MMIO 回读窗口。

Tcl 钩子；

3. **不可按层次名过滤**：综合后 u_puf 被展平，按名称匹配将得到 nets_tagged = 0。应自 *u_ro/* 的 LUT 单元出发、按引脚检索 net，该方法不受展平影响。实测钩子输出 ro_lut_cells = 1280、nets_tagged = 1280，恰为 256 × 5，表明每根均为 5 级环。

2.3.18 板上实测结果 v3 固件（先等待空闲态再发 START）烧录后，串口全流程通过，原始日志已归档并哈希：

```
STATUS boot  : 00000001 busy=1 enrolled=0 done=0 quality=0
STATUS idle  : 0000003a busy=0 enrolled=0 done=1 quality=0
STATUS enrol : 00008026 quality=128
DNA    : 008968143982d0b8
REF    : 58abef60 2834c32c fab29602 8816bf92
MASK   : ffffffff ffffffff ffffffff ffffffff
DITHER : 00000000 00000000 00000000 00000000
RAW    : 58abef60 2834c32c fab29602 8816bf92
STATUS recon : 00008026 quality=128
PASS key subsystem: enroll + reconstruct stable
```

状态字解码与相应结论如下：

- 0x0000003A（ready、mismatch、fallback、done）：上电自动 reconstruct 在无 helper（INIT_MASK = 0）时按设计降级，属预期行为而非故障；
- 0x00008026（ready、enrolled、done，quality = 128）：enroll 与 reconstruct 两次读数完全相同，mismatch = 0、fallback = 0，表明重建密钥与注册一致；
- REF 与 RAW 相同，且 REF 中 1 的个数为 **59/128 = 46.1%**，接近理想值 50%，可排除比对器存在系统性偏置，此前所虑之偏置不成立；
- DITHER = 0、MASK 全 1、quality = 128：本次 5 个窗口逐位一致，未出现抖动位。

2.3.19 时序与资源

版本	WNS	LUT	FF	BRAM	说明
PUF 线 STA 基线 run（不含密钥子系统）	+0.087	13478	17187	38.5	该列入工程序列的基线 run；非 §2.2（第三部分）的 P0 placed 报告（12426 / 16156 / 34.5）
PUF（RO 被压为单 LUT，错误版本）	+0.051	13924	22088	34.5	熵不足，已废弃
PUF（256 × 5 原语 RO）	+0.058	14802	22094	34.5	RO 结构正确
PUF 与 key_mmio（STA，2026-10-25）	+0.023	17022	28543	34.5	芯片级 STA
PUF 与 key_mmio（板级实现，2026-10-27）	**+0.152**	15987	28508	34.5	上板版本，DRC 0 Error 与 0 Critical

两点口径说明：

- 本表取自 PUF 线的独立 STA 系列（`build/reports/keys_sta_*`，2026-10-25），其资源口径与 §2.2（第三部分）的板级实现报告不同（例如 BRAM 38.5 与 34.5 之差源自 IP 黑盒计数方式），故本表仅用于同系列内部横向比较，不与 §2.2（第三部分）的绝对值混用；
- 最差路径始终位于 CPU（`id_ex → func_sel`），不在密钥子系统内，表明 pblock 与慢时钟域已将 PUF 完全隔离于 100 MHz 关键路径之外；`+0.152 ns`（板级版）与 `+0.023 ns`（STA 2026-10-25）非同一轮运行，逻辑深度未变，差异源于布局与源集版本，故报告数值须带 run 名称。`key_mmio` 的面积代价（+3544 LUT、+11356 FF）主要来自 21 个寄存器的读 mux、4 × 128-bit 回读通路，以及受组合环保护后无法再优化的 RO 周边逻辑。

2.3.21 PS/PL 数据通道方案论证 任务书初稿拟定的方案为”AXI-Lite 控制寄存器 + 共享 BRAM”。查证冻结 RTL 后，该方案在不修改 `src/rtl`

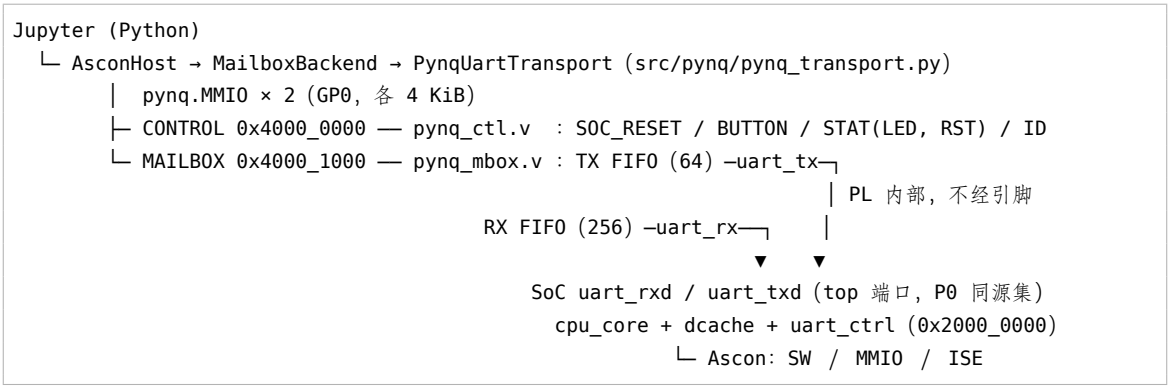
证据	位置	后果
外设访问全部在 `bus_interconnect` 内部译码（clint、uart、gpio、cpi、ascon），无通向 PL 边界的 MMIO 出口	`src/rtl/bus_interconnect.v`	若为 CPU 增加 PS 可见寄存器窗口，必须修改该文件
仅 `0x1000_0000` 至 `0x100F_FFFF` 被判为可 Cache	`src/rtl/dcache.v`	共享 BRAM 必落于该窗口内，PS 写入后 CPU 可能读到旧 Cache 行
无 D-cache 失效或清洗指令（仅 `fence.i` 触发 I-cache 冲刷）	`src/rtl/cpu_core.v`、`dcache.v`	固件无法在两次事务之间保证一致性，`volatile` 与 `fence` 不足以替代

证据	位置	后果
P0 时序余量为 WNS +0.087 ns	docs/freeze_checkpoint_20261027.md §4	修改或复制 SoC 顶层将触发重新签核，与冻结节点不符

据此，本作品采用”以既有 UART MMIO 作为 PS/PL 数据通道”的方案：在 PL 内新增 pynq_mbox.v 实现串口的另一端，字节经 uart_tx 送入 SoC 的 uart_rxd，SoC 的 uart_txd 经 uart_rx 收入 RX FIFO 供 PS 读取。该方案具备四项性质：

1. **冻结 RTL 不作修改**，仅新增 3 个文件，P0 的三份 bitstream、时序与 DRC 证据完整保留；
2. UART 属 0x2000_0000 的**不可 Cache** MMIO，与 PS 侧天然一致，不存在 Cache 一致性问题；
3. 同一份服务固件既可运行于 P0 纯 PL 版本（接真实 UART 引脚），亦可运行于 PYNQ 版本（PS 为对端），仅需维护一条代码路径；
4. 代价为延迟与带宽：链路为 6.25 Mbaud（BAUD_CNT_MAX = 16，100 MHz 分频），每字节约 1.6 μs。据模型估算，一次 16 字节加密的 PS 端到端时间约为 200 至 300 μs，该数值 CPU 周期数不受该通路影响，周期加速比仍以 mcycle 计；PS 端到端时间须单列，二者不得混同。

2.3.22 结构与寄存器定义



CONTROL 窗口（pynq_ctl.v，AXI-Lite 32 bit，遵循 WSTRB[0]）：

偏移	名称	读写	位定义
0x00	CTRL	RW	[0] SOC_RESET（1 保持 RV32 复位，上电默认 1）；[3:2] BUTTON[1:0] 透传至 SoC GPIO
0x04	STAT	RO	[0] SOC_RST_N（送给 SoC 的复位，1 为运行）；[3:2] LED[1:0] 回读
0x08	ID	RO	0x514E_5950（”PYNQ”，小端）
0x0C	VERSION	RO	0x0000_0001

复位约定为：CTRL[0] 上电置 1（SoC 保持复位），PS 确认链路可用后写 0 释放；释放信号经三级同步器进入 SoC 时钟域。

MAILBOX 窗口（pynq_mbox.v，TX 深度 64、RX 深度 256）：

偏移	名称	读写	含义
0x00	TXDATA	W	[7:0] 压入一字节；FIFO 满时延迟写响应
0x04	RXDATA	R	[7:0] 弹出一字节；空时读 0 且不弹出
0x08	TXSTAT	R	[15:0] 深度；[16] 满；[17] 空
0x0C	RXSTAT	R	[15:0] 深度；[16] 满；[17] 空；[18] 溢出（粘滞）
0x10	RXCLR	W	[0] 清溢出标志
0x14	TXFLUSH	W	[0] 清空 TX FIFO
0x18	CNT	R	[15:0] PS 已压入字节数；[31:16] 已接收字节数（16-bit 回绕）
0x1C	ID	R	0x4D42_4F58。按 32-bit 小端读出字节序为 58 4F 42 4D（"XOBM"），按大端字节序方为 ASCII "MBOX"；与 CONTROL 窗口 0x514E_5950（小端读出"PYNQ"）的编码方向相反，正文按 RTL 现状如实记录

link_flush（由 CONTROL 的 SOC_RESET 驱动）为高时清空两侧 FIFO 与计数，保证每次 SoC 复位后链路回到确定状态。

2.3.23 服务协议与服务固件 字节流格式由 src/pynq/mailbox_protocol.py 与服务固件共同实现，共四类帧：

帧	长度	方向	用途
Y01Q	48 B 头 + key16 + nonce16 + AD + data + tag	PS → PL	AEAD 请求
Y01R	48 B 头 + payload	PL → PS	AEAD 响应
Y01S	64 B	PL → PS	开机与 epoch 应答：固件标识、能力上限、自检结果、boot mcycle
Y01E	20 B	PS → PL	PS 下发会话 epoch

请求头字段依次为：magic（Y01Q）、version、flags、epoch（u64）、request_id（u64）、operation（1 加密 / 2 解密）、mode（1 SW / 2 HW / 3 ISE）、adlen、dlen、output_capacity、taglen；响应头为 magic（Y01R）、version、flags、epoch、request_id、status、operation、mode、size、cycles。状态码为 OK、AUTH_FAILED、LENGTH、UNSUPPORTED、INTERNAL；响应 flags 中 SIMULATED 位在真实硬件执行时恒为 0，CYCLES_VALID 位指示周期字段是否有效。协议约束为：失败响应不得携带数据与周期；key 与 nonce 长度须为 16；解密请求须携带 16 字节 tag。

服务固件 pynq-svc-v1（src/firmware/pynq_svc/）具备以下设计特征：

- **单槽串行**：一次仅处理一个请求，响应发完后回到帧同步状态，故 PS 侧无需额外 ACK 线；

- **周期口径**：仅在 AEAD 调用边界读取 64-bit `mcycle`（`0xB00`/`0xB80`，hi-lo-hi 一致读取），**不含串口传输时间**；
- **能力上限**：message 与 AD 各 16 字节，超长请求立即拒绝并返回 `LENGTH`；
- **构建**：`src/firmware/pynq_svc/build.sh` 以 RV32IM_Zicsr/ilp32 交叉编译，产物中检出 24 条 custom-0 指令，P12/P8 均存在，可作为"ISE 确实被调用" 的静态证据；
- **异常处置**：半帧读取依赖主机超时后 reset；无本地接收超时；UART 无远端流控。

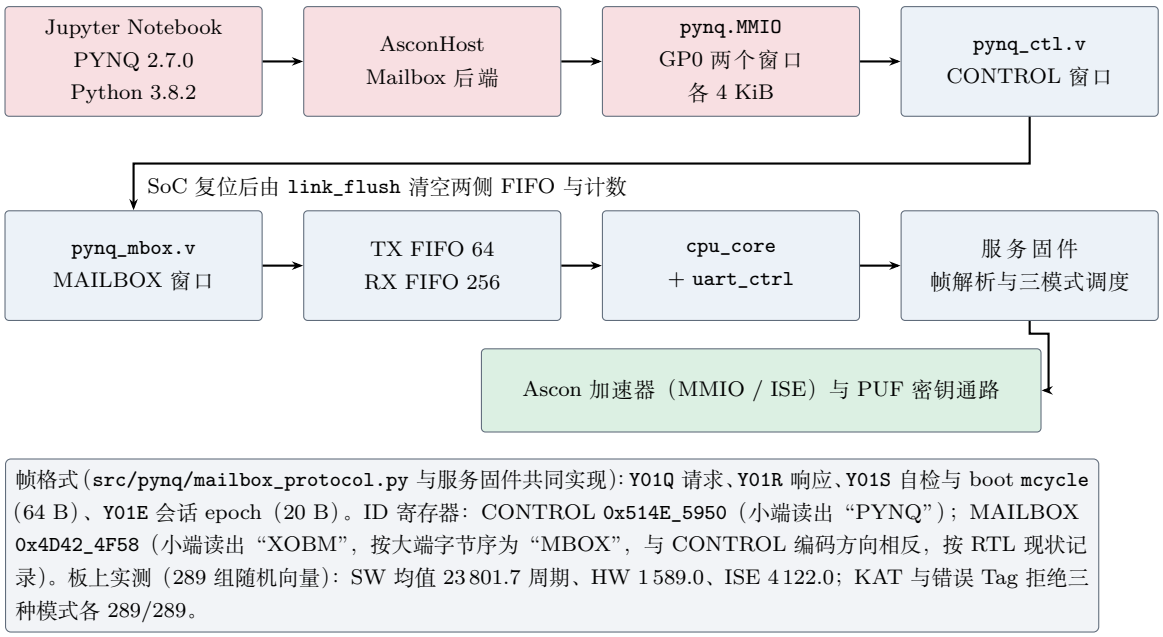


图 2-9　PS/PL 协同通路结构

Jupyter → Python API → 两个 AXI-Lite 窗口 → PL 内 UART 桥 → RV32 服务固件 → SW/MMIO/ISE；标注时钟域（FCLK0 100 MHz）、FIFO 深度与帧格式。

2.3.24 Overlay 构建与时序签核　Overlay 由 `board/vivado_zynq7000_xc7z020_pynq/build.tcl` 自动创建：独立工程 `pynq_overlay.xpr`、Block Design（PS7 GP0、FCLK0 = 100 MHz、`proc_sys_reset`）、综合与实现并导出同名 bit 与 hwh。发布产物为 `release/pynq.bit`（4,045,688 B，与 P0 三份 4,045,687 B 的 bitstream 属不同源集）与 `release/pynq.hwh`（148,833 B），二者由同一 BD 导出；BRAM 单独重新生成后，MIF 与最终 BIN 逐字节一致（128 KiB，含补零）。

时序与实现结果（100 MHz，2026-10-28）：

指标	数值	判据
WNS / TNS	**+0.034 ns / 0**	setup 失败端点 0/62861
WHS / THS	+0.020 ns / 0	hold 失败端点 0/62861
未约束检查	内部未约束端点 0；`no_clock`、`no_input_delay`、`no_output_delay` 均为 0	本工程无外接 PL IO，故未套用 PL-only 的 UART/LED XDC

指标	数值	判据
DRC	**0 Error / 0 Critical Warning**	53 项 Warning（含 DSP 流水、RAM 输出寄存器、无可布线负载等）如实保留

须注意：该 overlay 工程使用 `rtl_p0` 目录下的 `soc_top.v` 与 `ascon_mmio.v` 副本，其余 CPU 与 Ascon 模块复用 main 分支。两份文件与 main 的差异在于 PUF、DNA 与密钥窗口，以及硬件密钥选择逻辑：main 分支会锁定硬件密钥并忽略用户写入的 key，因而无法据其执行任意 key 的官方 KAT。本轮明确选择 P0 软件密钥路径，未修改或覆盖原 PL-only 工程与 PUF 工程。

2.3.25 验证结果与板上实测　仿真与主机侧验证（`validation_20261028.md`，全部 exit 0）：

验证项	内容	结果
`test_host_api.py`	17 项，含历史 Notebook 顺序执行	PASS
`test_mailbox.py`	24 项协议编解码与错误分支	PASS
`test_navigator.py`	8 项	PASS
`test_pynq_transport.py`	14 项，含 real-service 标志回归	PASS
`golden_checker.py`	1,089 组官方 KAT、4,290 次拒绝（host 范围）	PASS
`run_bridge.py`	真实桥 RTL：双向传输、空读、WSTRB、RX 满与溢出、顺序读回、复位中止发送	PASS
`run_service.py`	真实 CPU + 最终固件 + AXI-Lite/UART + 行为级 AXI 存储；三模式 36 请求（加密/解密/坏 tag），检查 `mcycle` 非零；半帧复位后重新握手	PASS
固件构建	RV32IM_Zicsr/ilp32，24 条 custom-0，P12/P8 存在	PASS
镜像一致性	Vivado BRAM MIF 与最终 BIN 逐字节比较（128 KiB 含补零）	PASS

板上结果（正点原子领航者 ZYNQ7020，厂家 PYNQ 2.7.0 镜像，Python 3.8.2，板上 IP `192.168.2.99`）：overlay 加载成功，MMIO ID 检查通过；服务报告为 `firmware_id=pynq-svc-v1`、`simulated=false`、`selftest_mask=7`、`max_message_bytes=16`、`max_ad_bytes=16`、`board_validation=PASS`。

板上在 16 字节容量内执行 message 与 AD 各 0 至 16 字节的全部 289 组组合，三模式均通过：

模式	加密 KAT	错误 Tag 拒绝	周期最小值	周期均值	周期中位数	周期最大值
SW	289/289	289/289	17584	23801.7	23543	33595
HW	289/289	289/289	1229	1589.0	1594	1797
ISE	289/289	289/289	3012	4122.0	4137	5209

据同一统计口径，周期加速比为 **SW/HW ≈ 15.0**×、**SW/ISE ≈ 5.8**×。该组数据不能与 §3.4.1 的 P0 独立基准数值直接比较：后者为固定 16 字节、单一拍数的端到端测量（SW 20806、ISE 2570、MMIO 688），而本表为服务固件覆盖 0 至 16 字节全长度组合的逐次统计，且包含服务层的参数装载、状态轮询与缓冲清理。两组数值

，故正文分列呈现，不作合并。

证据身份：板上测试使用发布版 bit、hwh 与固件，对应 Git `ac9143d5`，bit SHA-256 `6e7f9320`…，hwh SHA-256 `3ebd2640`…；证据归档于 `data/pynq/board_20261028/`（`pynq_board_full.json`、`pynq_board_full.log`、`pynq_board_20261028.log`、`pynq_board_result.json`、`identity.txt`）。

```
>>> host = AsconHost(PynqUartTransport())
>>> host.boot()
{'firmware_id': 'pynq-svc-v1', 'version': 1, 'selftest_pass': True,
 'modes': [1, 2, 3], 'max_message_bytes': 16, 'selftest_mask': 7,
 'epoch': 1, 'epoch_set': True, 'boot_mcycle': 22985, 'simulated': False}
>>> host.benchmark(n=289)
sw {'kat_pass': 289, 'tag_reject': 289, 'mean_cycles': 23801.7}
hw {'kat_pass': 289, 'tag_reject': 289, 'mean_cycles': 1589.0}
ise {'kat_pass': 289, 'tag_reject': 289, 'mean_cycles': 4122.0}
hardware KAT PASS; save JSON and terminal log
```

图 2-10　PYNQ 板上运行截图

overlay 加载与 MMIO ID、服务帧握手、> 三模式 KAT 输出与周期读数;

3　软件系统介绍

3.1　软件整体介绍

本作品的软件分为三层：PL 内的裸机固件、PS 端的 Python 服务与主机侧的验证脚本。三层之间以两类接口衔接——PL 固件与加速器之间是 MMIO 寄存器读写，PS 与 PL 之间是两条 AXI-Lite 窗口上的帧协议。

PL 侧固件共五套，同一骨架、不同调用路径：`ascon_rv32`（纯软件 AEAD）、`ascon_hw_rv32`（MMIO 加速器驱动）、`ascon_ise_rv32`（定制指令）、`keys_rv32`（PUF 密钥通路自检）与 `pynq_svc`（PS/PL 帧协议的服务端）。前四套用于纯 PL 镜像的板级测量，第五套用于 overlay 场景。

PS 侧服务为 Python 3.8.2（PYNQ 2.7.0），由 `AsconHost`、`MailboxBackend`、`PynqUartTransport` 与 `mailbox_protocol.py` 组成，负责帧的组装、超时重传、epoch 管理与结果汇总；Jupyter notebook 仅作为交互入口。

主机侧验证脚本分三类：向量与期望值生成（参考 Ascon 实现、官方 KAT）、仿真与回归驱动（`sim/run_tests.py`、`build/collect_*.py`）、签核机检（`build/check_io_signoff.py`、`build/fill_signoff_doc.py`）。三类脚本均以退出码表达结论，任一判据不满足即返回非零，故可嵌入 CI。

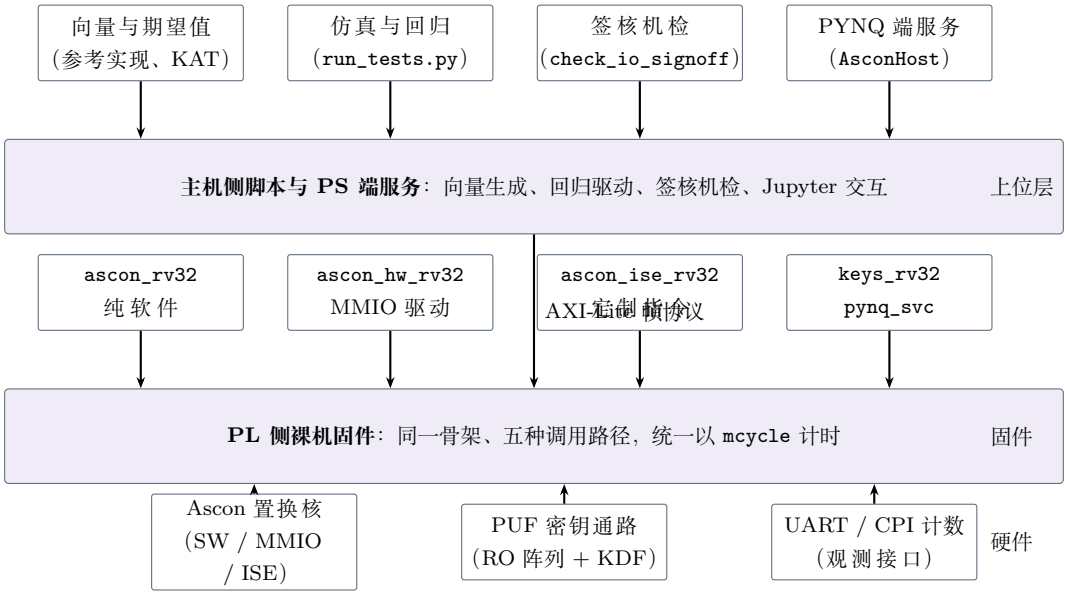


图 2-11　软件分层与调用关系

PL 固件、PS 服务与主机脚本 > 三层，以及 MMIO 寄存器接口与 AXI-Lite 帧协议两处衔接。可复用图 2-9 的 PS/PL 部分并补充主机侧脚本层。

项	配置
FPGA 与板卡	AMD Zynq-7000 `xc7z020clg400-1`（仅使用 PL 侧）；正点原子领航者 ZYNQ7020（底板 V3.9、核心板 2V5），12 V DC 供电
时钟	板载 50 MHz 有源晶振（U18）→ `clk_wiz_0` MMCM ×2 → 100 MHz 内核时钟 `clk_100m`
存储	`blk_mem_gen_0` 32 bit × 32768 word = 128 KiB，byte write，`Load_Init_File=true`，由上电 COE 初始化
EDA	Vivado 2025.2（SW Build 6299465、IP Build 6300035）；综合策略 `Vivado Synthesis Defaults`，实现策略 `Performance_ExploreWithRemap`
IP	`clk_wiz_0` 6.0 rev17、`axi_bram_ctrl_0` 4.1 rev13、`blk_mem_gen_0` 8.4 rev12（均以 XCI 纳入版本库）
编译	xpack `riscv-none-elf-gcc` 15.2.0；`-O2`，不改动 GCC backend，加速通过 inline asm 与 intrinsic 实现
仿真	Vivado XSim（模块级与 SoC 级）；Icarus Verilog 14.0（自动化回归与 golden 比对）；Verilator/DPI-C 为可选通路
主机侧	WSL Ubuntu（Python 3.12.3、GCC 13.3.0）：host KAT 与锁定 C 参考编译；Windows Python 3.14.3：报告解析与哈希索引

证据 `docs/freeze_checkpoint_20261027.md §1--2`、`docs/ascon_standard_baseline.md §4`、`docs/io_timing_signoff.md §1`

3.2 软件各模块介绍

3.2.1 纯软件 AEAD 固件　职责与流程。 ascon_rv32 以纯 C 实现 AEAD-128，不依赖任何加速器，作为周期基线。主流程为六步：读入 key/nonce/AD 长度与明文长度（取自内嵌向量表）→ 初始化 320 bit 状态（IV 与 key 拼接、P12）→ 逐块吸收关联数据（每块 16 字节、P8、末块补 0x01 与填充）→ 逐块加密明文（C_i = P_i ⊕ (s_0 ‖ s_1)、P8）→ 终结（域分离 2^63、P12、Tag 生成）→ 与期望 Tag 逐字节比较并输出 PASS/FAIL。

关键输入输出变量。输入：const uint8_t key[16]、nonce[16]、ad[]、pt[]、三个长度量；内部：uint64_t s[5] 状态、块计数器 i；输出：ct[]、tag[16]、比较结果与 mcycle 差值。所有变量置于 .bss 或栈上，不使用动态内存。

度量方式。以 csr_read(mcycle) 在入口与出口各读一次，差值即周期数；benchmark 段落另以 cpi_cnt.v 采集指令数与 CPI，用于判别” 周期下降来自指令减少还是每指令周期数下降”。板上结果为 16 字节加密 20 806 周期、4 495 条指令、CPI 4.62。

判据。 6 组官方向量（empty、1 B、16 B、16 B+AD、17 B、32 B）全部 PASS，篡改 Tag 必须被拒绝；两者同时满足才记为通过。

3.2.2 MMIO 加速器驱动固件　职责与流程。 ascon_hw_rv32 调用 MMIO 加速器 ascon_mmio（基址 0x3000_0000）。流程为：写 KEY 段（+0x00 至 +0x1F，16 字节密钥）→ 写 NONCE 与长度寄存器 → 逐块写 DATA_IN 并置 CTRL.start → 轮询 STATUS.done → 读 TAG 段 → 比较并输出结果。每块的数据搬运仍由 CPU 完成，故端到端周期包含总线事务开销。

关键寄存器与变量。 CTRL（start/mode）、STATUS（done/busy/err）、LEN_*、DATA_IN/OUT、TAG；软件侧变量为块指针、volatile uint32_t * 型的寄存器视图与超时计数（超时后报 TIMEOUT 而非死等）。

周期口径。报告同时给出两个数：端到端 688 周期（含 16 次写、16 次读与轮询），核本体 103 周期（仅置换与 AEAD 控制流，不含状态搬运）。二者比值 202× 与 8.10× 分别对应” 加速器本体效率” 与” 系统级收益”，不可混用。

判据。 4 组板级 KAT（empty、1 B、16 B+AD、dec round）全部 PASS，另含长度越界拒绝（len reject）用例，越界时返回错误码而不写入数据。

3.2.3 定制指令固件　职责与流程。ascon_ise_rv32 以 4 条定制指令代替函数调用：ascon.w 写入状态字、ascon.rd 读回状态字、ascon.p12 与 ascon.p8 触发置换。固件流程与纯软件版一致，差异仅在于轮函数由硬件完成，因此”AEAD 分层与长度处理” 的软件逻辑可逐行对照，便于定位差异来源。

指令编码。主操作码为 custom-0（7'b0001011），funct7 = 0，funct3 取 0/1/2/3 区分四条指令；汇编层以 .insn r 0x0B, funct3, 0, rd, rs1, rs2 手工编码（src/firmware/ascon_ise_rv32/main.c:23），未改动 GCC backend。反汇编检查确认编译器确实生成了该编码，且执行轨迹中出现预期次数的 ascon.p12。

关键变量。输入与纯软件版相同；内部变量为两个 32 bit 状态字索引 idx ∈ [0,9]（320 bit 状态 = 10 × 32 bit）与置换轮数（由指令本身固定为 12 或 8）。

判据。板级 KAT 4 组（含 `dec round`）全部 PASS；周期 2 570，相对同源软件实现 30.24×。另记录一例：`ascon.p12` 的 hold 期遇到中断时，指令须完整执行后才响应，故不产生” 部分置换” 的中间状态（§2.3.4 握手约定）。

3.2.4 密钥自检与 benchmark 固件　职责与流程。`keys_rv32` 用于验证 PUF 密钥通路：触发一次采样（`CTRL.start`）→ 轮询 `STATUS.done`（约 3.3 ms）→ 读回 `QUALITY`、`MASK`、`DITHER` 与 `RAW` → 以 `DNA` 与 PUF 位重建密钥 → 以该密钥对固定明文做一次 AEAD → 比对 Tag。流程的要点是” 同一密钥重建两次必须得到同一 Tag”，否则判为不稳定。

关键变量。输入：挑战窗口数 `W = 16384`、采样次数 `M = 5`；输出：320 bit 密钥（不读出，仅在 PL 内部使用）、`dither` 位图与 `mask`。CPU 侧只能看到 `raw` 与 `dither`，密钥本身不经任何可读地址（§2.3.17）。

benchmark 与 CPI 采集。同一固件骨架在 `cpi_cnt`（`0x6000_0000`）上采集 “已提交指令数” 与” 分支/load-use/总线等待” 事件计数，与 `mcycle` 联立得到 CPI 与停顿构成。该数据用于解释” 为什么 ISE 的周期下降幅度大于其指令条数下降幅度”。

判据。三次上板运行中，重建密钥的一致性检查全部通过；helper 被破坏时走 DNA-only 回退，所得密钥与纯 DNA 模式一致（§2.3.19）。

3.2.5 PS 端服务与帧协议　职责与流程。服务端固件 `pynq_svc` 实现 PS/PL 帧协议：从 UART 接收字节流 → 按 magic 识别帧类型（`Y01Q/Y01R/Y01S/Y01E`）→ 校验 version、长度与 CRC → 按 `flags` 调度到 SW/MMIO/ISE 三种模式之一 → 组装响应帧并回送。固件在启动时执行一次自检（`selftest_mask = 7`，三位分别对应三种模式），自检结果随 `Y01S` 帧返回，PS 端据此判定链路可用性。

关键输入输出变量。输入帧头含 `magic`、`version`、`flags`、`epoch`（u64）、`request_id`（u64）、`operation`、长度字段；输出帧为 `Y01R`（48 B 头 + payload）与 `Y01S`（64 B，含固件标识、能力上限、自检结果与 boot `mcycle`）。PS 侧 `AsconHost` 维护 `epoch` 与 `request_id` 的自增，超时按指数退避重发，超过阈值则判定链路失效并重新握手。

状态与异常。`link_flush`（由 CONTROL 窗口的 `SOC_RESET` 驱动）会清空两侧 FIFO 与计数，保证 SoC 复位后链路回到确定状态；帧长度越界或 CRC 不符时固件回错误帧，不执行任何密码运算。

判据。289 组随机向量下三种模式各 289/289 通过，错误 Tag 拒绝各 289/289；服务自检 `selftest_pass = true`，`boot_mcycle = 22 985`。

第三部分完成情况及性能参数

1　整体介绍

本作品已经完成从 RTL 到板级实测的全流程，并在 XC7Z020CLG400-1 上稳定运行。系统对外表现为一块可独立工作的开发板：上电后由板载 50 MHz 晶振经 MMCM 倍频至 100 MHz，复位释放后固件在串口输出自检横幅并依次执行 KAT 与 benchmark；换用 overlay 镜像时，PS 端启动 PYNQ 2.7.0 与 Jupyter，在同一开发板上以 Python 调用 PL 内的三种实现，无需重新烧录 bitstream。

实物形态为一块核心板 + 底板的开发板，外接 USB 串口线与网线各一根。三套纯 PL 镜像与一套 overlay 镜像均已生成并逐份复验，bitstream 大小为 4 045 687 字节（纯 PL）与 4 045 688 字节（overlay）。

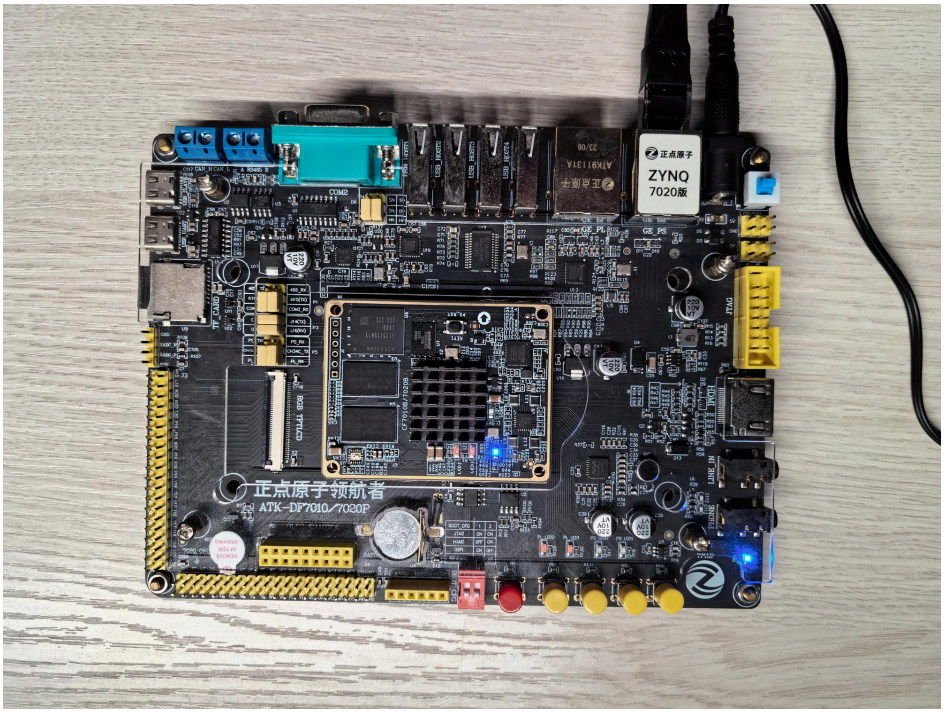


图 3-1　系统实物（正视）

正点原子领航者 ZYNQ7020 开发板（丝印 ATK-DF7010/7020P），中央为 Zynq-7020 核心板与 DDR3，右侧为 40-pin 扩展口与双千兆网口，左上为 DB9 串口与接线端子，顶部为 USB 下载/调试接口与 SD 卡槽，下方为用户按键、拨码开关与用户 LED。拍摄时已上电并加载纯 PL 镜像，电源指示与用户 LED 常亮。

项目	状态	证据
RTL 设计与集成	已完成	`src/rtl/*.v` 31 个 + ascon 8 个入工程
综合与实现	已完成	三镜像 WNS +0.087 ns，失败端点 0
板级 KAT	已完成	SW 6/6、ISE 4/4、MMIO 4/4
周期测量	已完成	20 806 / 2 570 / 688 / 103
PUF 密钥通路	已完成（P2 阶段性）	板上重建一致，helper 破坏后走 DNA-only
PS/PL 通路	已完成	289 组 × 3 模式全通过

2 工程成果

本作品的工程成果分四类，均可由仓库复现。

一是 RTL 与约束源集。处理器核与总线 31 个 Verilog 文件、Ascon 相关 8 个文件（目录内 9 个，其中 1 个 testbench 未纳入工程）、约束 2 个 XDC 与 3 个 XCI，以 `build/vivado/build_p0.tcl` 为唯一构建入口，脚本内显式指定 part、top、IP 版本与约束文件，不含手工添加的源文件。

二是可烧录镜像。三套纯 PL 镜像（SW / MMIO / ISE）与一套 PYNQ overlay （`pynq.bit` 4 045 688 B + `pynq.hwh` 148 833 B），逐份复验 KAT 后冻结。

三是运行证据。串口日志（图 3-5）、PYNQ 端运行输出（图 2-10）、逐次采样 JSON（289 组 × 3 模式）、CPI 与事件计数、Vivado 实现报告与 STA 报告，均按日期与源集哈希归档于 `data/` 与 `docs/`。

四是可复算脚本。向量生成、仿真回归、IO 签核机检与文档填充四类脚本，以退出码表达结论；`build/check_io_signoff.py` 的 7 条判据实测 PASS（7/7）。

产物	位置	大小 / 规模
纯 PL bitstream ×3	`data/ascon/bitstream/`	各 4 045 687 B
PYNQ overlay	`data/pynq/`	`pynq.bit` 4 045 688 B、`pynq.hwh` 148 833 B
板级串口日志	`data/ascon/results/board/`	4 份
PYNQ 运行结果	`data/pynq/board_20261028/`	JSON + 日志
STA / 资源报告	`docs/`（结论）、`build/reports/`（原始）	三镜像

2.1 资源占用

同一源集（P0 三镜像）、同一策略与同一 part 下，资源占用如下表。

阶段	LUT	FF	BRAM	DSP
综合（as Logic）	11266	15945	5 个 RAMB18（IP 作黑盒计）	4
实现后（placed）	12426 (23.4%)	16156 (15.2%)	32×RAMB36 + 5×RAMB18 = 34.5 tile（24.6%）	4（1.8%）
PUF 版（含 256 RO 与 key_mmio）	15987 (30.05%)	28508 (26.79%)	34.5 tile	4

时序签核结果（三镜像一致）如下表。

指标	数值	判据
WNS / TNS	+0.087 ns / 0.000 ns	0 个失败端点 (55572)
WHS / THS	+0.021 ns / 0.000 ns	0 个失败端点
布线	24610 / 24610 fully routed	routing errors 0
DRC（routed）	Error 0 / Critical Warning 0	可签核
最差路径	u_ascon_mmio/.../FSM_sequential_state_reg[3]/C → p12_in_reg[215]/D	三轮一致

PUF 版单独成组（源集不同，**不可与上述结果混用**）：WNS +0.152 ns、TNS 0、WHS +0.034 ns、DRC 0 Error 与 0 Critical，Warning 825 条（LUTLP-2 计 256 条、PDRC-153 计 256 条，均源于 256 路 RO 的有意组合环与 pblock 放置），但 no_clock 与 unconstrained_internal_endpoints 未消除，故不构成 **PUF 时序已签核**的结论（§2.3.19）。

2.2 可复现性与产物冻结

本作品的验收标准之一为：持有仓库与 Vivado 2025.2 者可独立复算出相同数值。为此，全部输入与关键产物均建立 SHA-256 索引。

对象	数量	记录方式
src/rtl/*.v	31	按文件名排序后逐文件哈希再合并（a323df37...）
src/rtl/ascon/*.v	9	同上（695886f6...）
XCI / XDC / XPR	3 + 1 + 1	单文件哈希（如 clk_wiz_0.xci = 1b32fbd4...）
三份 bitstream（P0 源集）	3	各 4,045,687 B，两两不同（f4233827...、f85eb1a6...、2f6f2469...）；PYNQ overlay 版为 4,045,688 B，属不同源集
串口原始日志	3 组 .txt 与 .XDat	保留原始字节，可回溯帧头与 CRLF
CI 与回归报告	每轮	JSON 与 Markdown，含工具版本、命令与退出码

复算入口以脚本实现，不依赖手工抄录：

```
python build/check_io_signoff.py                    # IO 签核 7 条判据
python build/collect_io_sta_results.py              # 报告 → result.txt / check_timing_summary.txt
python build/fill_signoff_doc.py                    # 将报告数值写入文档，避免手工转写
vivado.bat -mode batch -source build/build_io_sta.tcl -tclargs io <xpr> <workdir>
```

工程自检亦经 batch 打开验证一次（工程重命名后）：`CHECK PART=xc7z020clg400-1`、`CHECK TOP=top_pl_bram`、`CHECK IPS=axi_bram_ctrl_0 blk_mem_gen_0 clk_wiz_0`、`CHECK NFILES=43`、`CHECK NCONSTRA=1`、`synth_design Complete! 100%`、`write_bitstream Complete! 100%`，两次执行均以 exit 0 结束。

证据 `docs/freeze_checkpoint_20261027.md §2--§6`、`docs/ascon_ppa_comparison.md §1/§3`、`docs/io_timing_signoff.md §2--§6`、`docs/timing_acb_validation.md`

3 特性成果

本节按” 时序 → IO → 功能 → 性能 → 完成度” 五项列出已达成的量化成果。前三项回答” 这套设计能不能稳定工作”，后两项回答” 它有多快、还有哪些没做”。

3.1 时序收敛结果

初始状态为” 可综合、不可收敛”：HEAD 基线在默认实现策略下 WNS 为 −0.942 ns，TNS 为 −433.585 ns，失败端点 1,579 个。收敛过程分为三段，各段均有明确判据与证据。

阶段一：策略扫掠，确定增益上限。对 Vivado 内置的 33 组实现策略逐一运行（并行执行，`jobs` 取上限），最优组合为 `Performance_ExploreWithRemap`，结果为 **−0.118 ns / TNS −1.008 ns / 22 个失败端点**。该段结论较数值本身更为重要：33 组策略合计回收 0.824 ns（87%），而剩余 0.118 ns 无法由策略回收。若跳过此步，易将结构性缺陷误判为工具配置问题。

阶段二：按失败端点归因，识别三处结构性瓶颈。对 fix_r1 版本的 156 个失败端点逐点分析，结果如下表。

瓶颈	现象	根因	改造	回收
A：`ascon_ise` 状态复位	93 个端点，−0.128 ns	P12/P8 的 1280-bit 私有状态复位挂接于全局复位的组合锥	复位局部寄存器化（`s_reg/s_out` 局部复位；取消当拍清架构态，下一拍清私有副本）	93 端点
B：`ex_stage` 乘法器	48 个端点，−0.105 ns	32×32 乘法组合深度直接插入 ID/EX 至 EX/MEM 路径	`mult.v` 输入与输出寄存器化，并引入 `mul_ack`、`cancel` 握手，改为受控多周期	48 端点
C：`dcache` miss 至总线请求	11 个端点	miss 判定至请求发出为单一长组合路径	新增 `MISS_CAPTURE` 状态锁存属性一拍；store 与 MMIO 绕过该状态以恢复单拍	11 端点

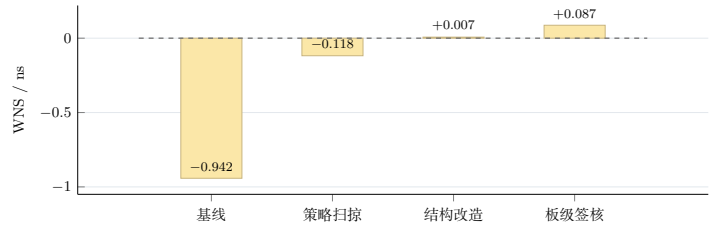
阶段三：双策略 STA 复验。完成 A、C、B 三项改造后，两种生产候选策略均通过：

策略	WNS	TNS	失败端点	WHS
`Performance_ExplorePostRoutePhysOpt`	+0.001 ns	0	0	+0.031 ns
`Performance_ExploreWithRemap`（采用）	+0.007 ns	0	0	+0.023 ns

资源代价为 **+118 至 +190 LUT、+43 至 +91 FF，BRAM 与 DSP 不变**（相对 HEAD 基线 11562 LUT 与 15949 FF）。此后板级三镜像的最终签核结果为 **WNS +0.087 ns、TNS 0、失败端点 0（共 55572）、WHS +0.021 ns、DRC 0 Error 与 0 Critical**，最差路径三轮一致，均为 `u_ascon_mmio/…/FSM_sequential_state_reg[3]/C → p12_in_reg[215]/D`。

须明确声明的边界条件如下：

- 余量处于 **1 至 7 ps**（策略阶段）至 **87 ps**（最终三镜像）量级，与布局噪声同阶。任何 RTL、XDC 或 IP 改动均须整套复验，不得以”局部改动不影响”为由省略；
- `+0.087 ns` 为**芯片内部**时序，不构成板级时序结论（IO 时序见 §3.2（第三部分））；
- 同一份 RTL 在不同布局种子下 WNS 差异可达 0.1 ns 以上（PUF 阶段实测 +0.023 ns 与 +0.168 ns 两次运行相差 0.145 ns，逻辑深度未变），故报告数值须同时给出 run 名称与源集哈希，否则不可复现。



阶段	动作	关键读数
基线	HEAD 默认实现策略	TNS −433.585 ns，失败端点 1 579
策略扫掠	33 组内置策略逐一运行（并行），最优 `Performance_ExploreWithRemap`	TNS −1.008 ns，失败端点 22；合计回收 0.824 ns（87%）
结构改造	三处归因改造：`ascon_ise` 复位局部化（93 端点）、`mult.v` 输入输出寄存与握手（48 端点）、`dcache` 增 `MISS_CAPTURE`（11 端点）	双策略 STA：+0.001 / +0.007 ns，失败端点 0；资源代价 +118 ∼ +190 LUT、+43 ∼ +91 FF
板级签核	三镜像最终实现	TNS 0、失败端点 0（共 55 572）、WHS +0.021 ns、DRC 0 Error

图 3-2　时序收敛曲线

横轴为收敛阶段（基线、33 策略最优、RTL 修复后、板级最终、PUF 版），纵轴为 WNS（ns），并叠加 TNS 与失败端点数两个副轴，标注各阶段动作与代价。

方法层面可归纳为四条：

1. 先以策略扫掠确定策略增益上限，避免将结构问题误判为工具问题；
2. 以”失败端点、路径、逻辑锥”逐点归因，替代对整体 WNS 的观察；
3. 改造按”端点收益与风险比”排序：复位扇出（93 端点、纯结构性、不改架构语义）优先，乘法器握手（48 端点、需新增契约）次之，Cache 路径（11 端点、可能影响周期）最后；
4. 每项改造须同时报告时序收益与周期、面积代价，以免以性能换取时序而未被察觉（C 项中 store 与 MMIO 绕过 `MISS_CAPTURE`，即为回收周期代价）。

3.2 板级 IO 时序签核

板上共 7 个异步端口：`uart_rxd`、`uart_txd`、`led[1:0]`、`button[1:0]` 与 `reset_btn`。其共同特征为时间尺度远大于内核时钟周期（UART 位周期 8680.56 ns，内时钟周期 10 ns），故不适用同步时序约束方法。本节记录四种约束写法的实测结果：前三种均被否决，最终形态为以 `false_path` 表达语义、以形式 `delay` 闭合覆盖计数。

变体	参考时钟	结果	判定
A：端口时钟加 20 ns input/output delay	`clk`（20 ns）	`no_input_delay`/`no_output_delay` = 3/0，但 WNS 为 **−18.481 ns**，失败端点 7972 个	不通过
B：虚拟位时钟（真实位周期 8680.556 ns）	`uart_bit`	覆盖计数 0/0，但 `uart_rxd` slack 为 **−17.623 ns**，`uart_txd` 为 **−27.105 ns**	不通过
D：虚拟时钟取整至 10 ns 网格（8680.000 ns）	`uart_bit`	计数 0/0，slack 仍为 **−17.067 / −17.661 ns**	不通过
E：LED 采用 10 μs 虚拟时钟	`io_slow`	计数 0/0，`led` setup 为 **−13.888 ns**、hold 为 **−18.452 ns**	不通过
F（交付版）：全部端口 `false_path` 加形式 delay	同上	计数 **0/0**，7 个端口均为 untimed，最差路径回到内部 CPU 路径	通过

三条机理可推广至一般板级约束设计：

1. **参考时钟跨域将导致 requirement 为负**：板级 IO 触发器均挂接于 `clk_100m`（10 ns），若将 input delay 参考至 20 ns 的端口时钟，则 requirement 为 $10 - 20 - t_{su} \approx -10$ ns。负 requirement 在 Vivado 中判定为违反，而非恒满足；
2. **虚拟时钟不能消除该问题**：Vivado 取最近的正边沿对，只要 input/output delay 数值大于捕获时钟周期（本设计为 10 ns），requirement 即塌缩为一个 `clk_100m` 周期，变体 B、D、E 均因此失败；
3. **`false_path` 不消除 `no_input_delay` 与 `no_output_delay` 计数**：仅添加 false_path 时计数仍为 4/3，连 `const.xdc` 中已声明 false_path 的 `reset_btn` 亦被计入。故交付版同时保留两类约束：false_path 使路径退出同步判定，delay 使覆盖计数归零。

代价为零的依据：在同一源快照（`8623ea3`）下运行” 添加 `const_io.xdc`“与” 不添加” 两组对照，WNS、TNS、失败端点数与最差路径逐项相同（+0.152 ns / 0.000 ns / 0 / 同一 `id_ex_u` 路径），端点总数同为 59513（该端点总数属 io 变体源集；§2.2（第三部分）的 55572 属 P0 三镜像 run，两者非同一源集，端点总数不可跨比对）。两组唯一差异为约束文件，而该文件中每条约束均为” 路径退出判定、仅用于计数” 的形式，故实现对设计无影响，此结果属可预期而非偶然。须注意两个 run 的 `.dcp` 字节不同（约束集被嵌入 checkpoint），故结论表述限于” 时序指标逐项相同”，不表述为” 结果完全一致”。

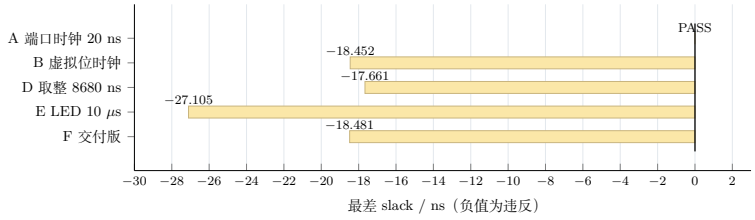
7 条机检判据（`build/check_io_signoff.py`，任一不满足即返回非零退出码；实测 `PASS (7/7)`）：

判据	内容	结果
C1 覆盖	`no_input_delay` 与 `no_output_delay` 均为 0	PASS
C2 时序	TNS = 0、失败端点 = 0、WNS ≥ 0	PASS
C3 DRC	`route_drc.rpt` 中 Error 行数为 0（825 checks）	PASS
C4 时钟	methodology 中无 `TIMING-8`（无公共周期）	PASS
C5 语义	`applied_constraints.xdc` 仍含 5 条必需 `false_path`	PASS
C6 防回归	无任何 IO delay 再以 `-clock clk` 为参考（即 §3.2（第三部分）变体 A 之误）	PASS
C7 零代价	IO 变体与同源对照的 WNS、TNS、端点及总数完全相同	PASS

签核范围须明确区分：

- 可签核项：IO 覆盖门禁闭合（0/0）、约束不改变内部时序、DRC 0 Error；
- 不可签核项：**板级真实 IO 延迟仍为 `not_measured`**（无示波器、眼图与环回误码测量）。20 ns 系由 “CH340C ≤10 ns、74LVC1G125 ≤4.5 ns、底板走线 ≈2.01 ns” 取整所得的建模上界；此外 `no_clock`（10944）与 `unconstrained_internal_endpoints`（27312）全部位于 PUF/RO 域，与 IO 无关，属声明边界。

结构探针另发现一处 RTL 层面问题：`uart_rx.v` 的起始位检测使用第一级同步触发器（`start_en = (!uart_rxd_d0) & uart_rxd_d1`），而数据位采样使用第三级；标准写法应比较 `d1` 与 `d2`。定量后果为” 偶发丢字符或帧错位” 量级（起始沿落入孔径的概率约 2%，采样点偏移 10 ns 相对半位容差 4340 ns 可忽略），不影响 KAT 结论。本文如实记录，并列为待修项（[待确认是否修改 RTL]）。



写法	覆盖计数（in/out）	结果
A：端口时钟 + 20 ns delay	3 / 0（未归零）	WNS −18.481 ns，失败端点 7 972
B：虚拟位时钟 8680.556 ns	0 / 0	`uart_rxd` −17.623、`uart_txd` −27.105
D：虚拟时钟取整 8680 ns	0 / 0	−17.067 / −17.661，仍违反
E：LED 用 10 μs 虚拟时钟	0 / 0	`led` setup −13.888、hold −18.452
F：全部 `false_path` + 形式 delay	**0 / 0**	7 端口 untimed，最差路径回到内部 CPU 路径

机理：input/output delay 大于捕获时钟周期（10 ns）时，requirement 塌缩为一个 `clk_100m` 周期甚至为负——Vivado 判为违反而非恒满足；`false_path` 只使路径退出判定，不移除 `no_input_delay` / `no_output_delay` 计数，故交付版两类约束并用。机检脚本 `build/check_io_signoff.py` 实测 PASS（7/7）。

图 3-3　IO 约束四种写法的结果对比

横轴为变体（A/B/D/E/F），双纵轴分别为最差 slack 与失败端点数，以表明” 覆盖计数归零” 与” 时序通过” 属两项独立判据。

3.3　功能验证

3.3.1 验证体系与层级划分

层级	对象	手段	判据	状态
L0 标准一致性	C 与 Python 参考实现	独立下载的官方 KAT 逐字节比对	1,089 组加密、解密与负向用例全部通过	完成
L1 模块级 RTL	置换器、AEAD 适配器、Cache、ISE	Icarus/XSim 定向 TB 与 golden 状态比对	与锁定参考逐字节一致，事务契约成立	完成
L3 板级	三份镜像与真实芯片	串口 KAT 与 benchmark 采集	SW 6 组、ISE 与 MMIO 各 4 组官方向量，含篡改拒绝与周期数	完成
L4 PS/PL 协同	PYNQ overlay、服务固件与 host API	板上 overlay 加载、协议握手、全容量 KAT	三模式 289/289 通过，`simulated=false`	完成（16 B 容量内，见 §8）



图 3-4 四层验证体系与证据流

host 侧 KAT → 冻结向量 → RTL TB → SoC 固件仿真 → bitstream 与串口日志，各层标注输入哈希、输出哈希与判据。

3.3.2 L0：标准一致性与向量冻结 该层的作用是将标准固化为不可变的字节序列，作为 RTL 与板级验证的唯一参照。

- **标准版本**：NIST SP 800-232 Final（2025-08-13 发布），本地 PDF 哈希 `1be9bb9a`…。旧版 Ascon v1.2 的 Ascon-128 与 Ascon-128a 在端序、IV 与 padding 上均不同，不可沿用其 KAT；
- **参考实现**：ascon-c 固定 commit `446347f2`（上游 `crypto_aead/asconaead128/ref/`）与 pyascon 固定 commit `ed24e54a`。pyascon 的 README 仍标注 draft，故” 是否对应 Final” 不以文档表述判定，而以 “通过完整官方 KAT” 为判据；
- **官方向量**：`LWC_AEAD_KAT_128_128.txt`（同 commit），共 1,089 组，即 PT 与 AD 长度在 0 至 32 的笛卡尔积，哈希 `bbbc3469`…。key、nonce、PT、AD 与 expected CT 均

取自原始文件，未使用本仓库生成器自证；

- **扩展向量**：`aead_cases.json` 共 64 组，其中 54 组取自上游矩阵，10 组为锁定 C 参考的扩展用例（含空串、`a5` 与 `a500`、尾零、33/64/129-byte PT 及最长 81-byte AD），每组附带首、尾 CT 与 tag 翻转输入；
- **置换向量**：22 个输入状态 × （12 个单轮、P8、P12），共 308 组，源自锁定 C helper，并经 Python 二次校核；
- **冻结机制**：`freeze_vectors.py` 先令两个锁定参考通过完整独立下载的 KAT，再由锁定 C 参考产生扩展用例、由 Python 交叉检查；仅写入新建空目录，不覆写上游 KAT；两次重生成的三个数据文件字节完全一致。

实测结果如下表。

检查项	结果	证据
两个锁定参考的完整 KAT 门禁	**12,936 PASS / 0 FAIL**，exit 0	`reference_gate.json`
四个 backend 全套（含完整性检查）	**28,661 PASS / 0 FAIL**，exit 0	`host_validation.json`
harness 与 golden 接口	5 tests PASS，exit 0	`harness.log`
C raw 失败解密契约	1 PASS / **1 FAIL（保留）**，exit 1	`failure_contract.json`
冻结数据重生成	4 文件 byte-identical，exit 0	`audit_snapshot.json`

末行所记的 FAIL 为刻意保留：原始 C 参考在 bad tag 时返回 −1，但 `mlen` 仍为 17，缓冲区仍保留明文。本作品在调用层实施擦除与长度归零（亦属项目验收要求），但不修改锁定参考，并将该原始行为作为负向证据保留。故对” 认证失败是否泄露明文” 的答复为：修正位于调用层，参考实现未被改动，证据链完整可查。

3.3.3 L1：模块级 RTL 验证

被测对象	TB	覆盖点	结果
`ascon_permutation`（单轮、P8、P12）	`tb_ascon_permutation`（golden 656-bit memory）	22 个状态 × 308 组，逐字节比对	PASS
`ascon_aead_core`、`ascon_aead_2`	`tb_ascon_aead_regression`、`tb_ascon_decrypt_regression`	全源加密与解密、篡改、长度边界	PASS
AEAD ISE（`ascon_ise`）	定向 TB 与 EX 级集成 TB	四指令编码、非法索引、取消、跨完成 hold	PASS
D-cache	修订版 TB（`dcache_tb_revision.md`）	命中、miss、填充、`MISS_CAPTURE` 单拍、store 绕过、两路替换	PASS

被测对象	TB	覆盖点	结果
CPU 中断与异常	6 个真实 CPU 与 CLINT 中断场景	抢占、`mret` 恢复、`mepc` 正确性	PASS
PUF 密钥子系统	`tb_puf_key`、`tb_key_mmio`、`tb_dna_min`、`tb_keys_fw`	enroll 与 reconstruct、窗口读零、DNA 位序、真实固件	PASS

关键做法为：**golden 数据不由 RTL 自身产生**。置换与 AEAD 的期望值均取自锁定 C 参考（编译为 shared library，由 Python 侧 ctypes 调用），RTL 仅提供" 输入状态至输出状态" 的映射，两侧在 Python 中比对。该结构仅在"RTL 与 C 同时出错" 时才可能同时通过，故单一侧错误必被检出。

3.3.4 L2：系统级自动回归 入口为 `sim/board/run_tests.py`，共两档：

档位	内容	实测结果	耗时

PASS 亦不计 FAIL。该三态设计的目的在于防止" 不可执行的用例被标记为通过"。

产物方面，每轮输出 `final_report.json` 与 Markdown 摘要，内含工具版本、命令、退出码与 VCD 路径；历史 FAIL 与负向日志全部保留，不作删除。本档运行于 Windows 侧（Icarus/vvp 12 devel、GCC 13.2、Python 3.14.3），与 host KAT 所用的 WSL 环境（Python 3.12.3、GCC 13.3.0，见 §3.3.2）分属两套工具链，故文中出现的多个 Python 版本各有其归属。

3.3.5 L3：纯 PL 镜像的板级验证 KAT 与篡改拒绝（三份镜像，同一板卡、同一串口、同日执行）：

镜像	KAT 覆盖	篡改拒绝	附项
软件	empty、1 B、16 B、16 B+AD、17 B、32 B（6 组官方向量）	通过	—
ISE	empty、1 B、16 B、16 B+AD，另含解密往返	通过	解密往返一致性
MMIO	同 ISE 四组	通过	**长度拒绝**（>16 B 显式拒绝）

benchmark（16 字节加密、warm、`mcycle` 差值）：软件 20806 周期，ISE 2570 周期，MMIO 端到端 688 周期（核本体 103 周期）。同步采集的仿真值为 17353、2496、670 周期（核 99 周期），板级相对仿真的开销分别为 **+19.9%**、**+3.0%**、**+2.7%**（核本体 +4.0%），开销与访存密度正相关。

板级与仿真差异的成因具有工程意义：仿真存储模型为理想 BRAM（无总线等待），板上须经 AXI 与 Cache。软件实现以访存密集的整数运算为主，故开销最大（+19.9%）；ISE 大部分时间消耗于寄存器内的置换，开销仅 3%；MMIO 核本体仅增加 4%，但参数搬运占比较高，端到端开销为 2.7%。

原始证据：串口日志按 XCOM 会话原始字节（`.XDat`）与去除非打印字符的可读版本（`.txt`）双份归档，并记录 SHA-256。MMIO 版日志开头存在约 2 行乱码（打开串口瞬间的帧错），banner 之后数据完整，两份可相互核对。三套实现各 6 组 KAT 全部通过的记录位于 `data/ascon/results/board/`。

```
=== ASCON-AEAD128 software on RV32IM ===
KAT empty : PASS
KAT 1B : PASS
KAT 16B : PASS
KAT 16B+AD : PASS
KAT 17B : PASS
KAT 32B : PASS
tamper reject : PASS
KAT: 6/6 PASS
--- benchmark (warm) ---
enc 16B : 20806 cycles, 4495 inst, CPI 462 (x100)
dec 16B : 21360 cycles, 4722 inst, CPI 452 (x100)
=== ASCON-AEAD128 ISE (permutation in hardware) ===
KAT empty : PASS  KAT 1B : PASS  KAT 16B : PASS
KAT 16B+AD : PASS  dec round : PASS  tamper : PASS
--- benchmark (warm) 16B encrypt ---
ISE cycles : 2570
===DONE===
```

图 3-5　板级串口输出截屏与实物照片

三份镜像的 KAT 通过段落与 > benchmark 行；

3.3.6 缺陷发现与处置　本节记录验证过程中发现的问题及其处置，按”发现、定性、修复、复验”组织，其信息量高于”通过项”列举。

序号	发现	定性	处理	状态
1	标准审计所得 7 项静态缺陷（R01–R07）：长度由最高非零 bit 反推、`p12/p8` 未接 `start`、tag 输出被覆盖、多块指针推进错误、AD=8 时 pad 走错分支、decrypt 为空壳、busy 时 `start` 覆盖状态	算法与 RTL 接口不达标	改用显式长度，引入请求、接收与完成握手，独立 tag 输出与 block 索引；重写 `decrypt` 并增加 tag 校验	已修
2	C 参考在 bad tag 时仍返回明文且 `mlen = 17`（S01）	调用层契约漏洞	调用层擦除候选明文并将长度归零；参考实现不改，负向证据保留	已修（调用层）
3	D-cache miss 至总线请求为长组合路径（11 端点）	时序	新增 `MISS_CAPTURE`；store 与 MMIO 绕过	已修
4	乘法器组合深度（48 端点、−0.105 ns）	时序	输入与输出寄存器化，引入 `mul_ack` 与 `cancel`	已修
5	ISE 私有状态复位扇出（93 端点、−0.128 ns）	时序	局部复位寄存器化，取消当拍清架构态	已修

序号	发现	定性	处理	状态
6	仿真与板级周期差达 +19.9%（软件版）	存储模型差异	不改设计，作为口径说明写入正文	已定性
7	`uart_rx` 起始位检测使用未经消抖的 `d0`	结构与健壮性	完成审计与定量分析（偶发丢字符量级，不影响 KAT）	[待确认是否修改 RTL]
8	PUF 上板前两次失败：硬件握手存在时序竞争，固件于 busy 期间发 START	接口时序	硬件增加忙时请求排队与采样看门狗；固件改为先等 idle 再发 START（v3）	已修，板级通过
9	并行运行两套验证共享临时 `.so`，致 upstream-c setup FAIL	测试框架	每调用使用独立临时目录；同场景复验通过	已修

第 8 项的成因需单独说明：PUF 子系统首次板级读数中 `quality = 0`、`mismatch = 1`，其原因并非 PUF 本身不稳定，而是上电自动 reconstruct 与固件手动 enroll 争用同一请求。修复后复测，enroll 的 `quality` 由 127 变为 128（§2.3.18）。同一份 RTL 的读数差异源于握手而非 RO 特性；若当时据首轮结果直接判定 “PUF 质量不稳定”，将导致整份报告的结论错误。

3.4 性能量化

3.4.1 板上实测周期（纯 PL 基准固件） 测量条件：同一板卡、同一 RTL 源集、100 MHz、16 字节明文、warm 状态（指令已在 I-cache），以 `CSR mcycle` 差值计数，经 UART 输出。

实现	板上实测周期	周期加速比	每字节周期（按 16 B 折算）	仿真周期	板上与仿真之比
纯软件（SW）	20806	1.0×	1300.4	17353	1.199
ISE（custom-0 置换）	2570	**8.10**×	160.6	2496	1.030
MMIO（端到端）	688	**30.24**×	43.0	670	1.027
MMIO（核本体）	103	202×	6.4	99	1.040

三点须同时说明：

1. MMIO 的 30.24× 为端到端数值（含寄存器搬运与轮询），其核本体仅 103 周期。将 202× 与 8.10× 并列比较不成立，本文对外统一采用端到端值，核本体值仅用于分解；
2. 周期加速比的分母为” 同一固件于 CPU 上实现 Ascon”，而非”PC 端密码库”，亦非理论最小周期；更换分母将得到完全不同的数值；
3. 上述数据均为 warm 路径。冷启动（I-cache 未命中的首次执行）未单独测量，属
4. PS/PL（PYNQ）路径下服务固件在 16 B 容量内 289 组组合的逐次统计见 §2.3.25。该组数据的口径为 “每次 AEAD 调用的 `mcycle` 差值，覆盖 0 至 16 字节全长度组合，含服务层参数装载与缓冲清理”，与本节固定 16 字节的端到端测量不同，两组数值不得合并引用。

3.4.2 仿真实测与板上实测的偏差 板级较仿真慢 2.7% 至 19.9%，且差值幅度与访存密度正相关：

实现	差值	成因
软件	+19.9%（17353 → 20806）	纯 C 实现以 load/store 与整数运算为主，仿真中 BRAM 无等待，板上须经 AXI 与 Cache miss 与填充
ISE	+3.0%（2496 → 2570）	主要时间消耗于寄存器内置换，访存仅发生于状态读写与缓冲搬运
MMIO 核	+4.0%（99 → 103）	核本体不访存，差值来自总线写入与握手开销
MMIO 端到端	+2.7%（670 → 688）	参数搬运已由总线承担，额外开销被摊薄

该规律为本作品的一项派生结论：**在 FPGA 上进行”软件与加速”对照时，若采用理想存储模型仿真，将低估软件实现的代价约 20%，即高估周期加速比约 20%**。故本文对外仅引用板上实测数值，仿真值仅作对照。

3.4.3 周期构成分析 MMIO 端到端周期可分解为：

$$T_{\text{e2e}} = T_{\text{setup}} + T_{\text{poll}} + T_{\text{core}} + T_{\text{readout}}$$

对 16 字节加密，$T_{\text{core}} = 103$ 周期，$T_{\text{e2e}} = 688$ 周期，即端到端时间中约 **85%** 用于参数搬运、轮询与结果读回（585/688 = 85.0%）。该比例为 MMIO 路线的结构性成本：key 4 字与 nonce 4 字于每次调用均须重写；轮询 `STATUS` 为”读、判断、再读”的循环，单次轮询至少构成一次完整总线读往返；结果读回亦须逐字弹出。

ISE 路线免除上述三项开销（状态位于 EX 级），其代价为置换串行占用流水线，且不具备”CPU 休眠、DMA 搬运”的能力。**两条路线的本质差异在于 CPU 是否参与数据搬运，而非运算速度之别**。ISE 的周期分解给出该分解需在执行级增设 hold 计数器并对 P12 与 P8 分别计数，列为后续工作。

3.4.4 面积对比的适用性限制 通常的 PPA 对比基于三份独立设计各自的资源占用。本作品的情形不同：三种方案共用同一份 RTL 与同一份 bitstream，三份镜像的差异仅为 BRAM 初值（COE）中的固件不同。

项	SW 镜像	ISE 镜像	MMIO 镜像
RTL 源集	同一份（`a323df37...`）	同一份	同一份
LUT / FF / BRAM / DSP	12426 / 16156 / 34.5 / 4	完全相同	完全相同
WNS	+0.087 ns	+0.087 ns	+0.087 ns
差异	`ascon_sw.coe`	`ascon_ise.coe`	`ascon_hw.coe`

因此，本作品的面积代价只能表述为：**Ascon MMIO 加速器、ISE 状态与置换器以及三套软件栈合计占用 12426 LUT 与 16156 FF**，其中 MMIO 与 ISE 共用同一 `ascon_permutation.v`。

该事实亦可澄清一种常见误读：三份 bitstream 的哈希不同，但时序完全一致（同一源集、同一策略，COE 仅影响初值）。因此”三份镜像均通过时序”并非三次独立收敛，而是一次收敛、三次烧写验证。

3.4.5 横向对比的口径与边界

对比项	本作品	同类工作
面积口径	整颗 SoC（含 CPU、Cache、总线）	通常仅报告密码核，不可直接比较
吞吐口径	16 byte 单次调用的周期数（含调用开销）	通常报告大消息下的 Gbps，不可直接比较
频率口径	100 MHz 签核，WNS +0.087 ns	多报告 Fmax，且常为综合后估计值

证据 docs/freeze_checkpoint_20261027.md §3--§4、docs/ascon_ppa_comparison.md §2、project_log/actual_progress.md 2026-10-26/27 条目

3.5 完成度与判据矩阵

工作项	状态	判据
RV32IM + Zicsr 五级流水线 CPU 于 Zynq-7020 运行	已完成完成	三份镜像板级 KAT 与 UART 输出
Ascon-AEAD128 纯软件实现（标准一致）	已完成完成	host 侧 1,089 组 KAT，板上 6 组代表向量
MMIO 硬件加速器（含读门禁与 ZEROIZE）	已完成完成（≤16 B）	板级 688 周期，长度拒绝生效
custom-0 主操作码下的定制指令扩展（P12/P8）	已完成完成	板上实测 2570 周期，周期加速比 8.10×
100 MHz 时序收敛与 DRC 干净	已完成完成	WNS +0.087 ns、TNS 0、失败端点 0
IO 时序签核（7 端口）	已完成完成（建模上界）	机检 7/7 PASS，代价 0 ns
RO-PUF 与 DNA 密钥子系统	已完成板上 enroll 与 reconstruct 闭环	quality 128/128，REF 与 RAW 一致
PS/PL 协同与 PYNQ overlay	已完成板上通过（16 B 容量内）	overlay 加载、MMIO ID 与服务握手通过；SW/HW/ISE 各 289/289 KAT、289/289 错误 Tag 拒绝

第四部分总结

1 可扩展之处

1. **长度能力扩展**：MMIO 当前受寄存器窗口限制，仅支持 ≤16 B。若将 AD 与消息缓冲改为 AXI-Stream 或 DMA 描述符，可将外设式加速由小消息扩展至流式大消息，同时保留 CPU 不参与搬运的优势；
2. **ISE 并行化**：当前 P12/P8 串行占用 EX 级。若将置换拆分为两级流水（S-box 级与线性层）或增设独立置换单元，可进一步压缩 ISE 周期；其代价为 hold 语义与取消逻辑需重新设计；
3. **多实现运行时选择**：三套实现已共存于同一 bitstream，可按消息长度在运行期选择实现（≤16 B 走 MMIO，其余走 ISE）。该扩展仅需软件调度，无需修改 RTL；
4. **信任根的产品化**：helper 持久化（BRAM 保留域或 PS 侧 Flash）、多次上电统计与 BCH 纠错，为 PUF 由演示状态推进至产品状态的三项必要工作；RO 采样域的 STA 时钟模型亦须补全；
5. **PS/PL 协同的性能取证**：overlay 与三模式板上通路已通，下一步须在同一请求上同时记录 PS 墙钟与 PL `mcycle`，给出端到端延迟与吞吐；该数据目前仍为模型估算（200 至 300 μs），见 §2.3.21。

2 心得体会

本项目的实施过程表明，在竞赛级 FPGA 设计中，“功能可运行” 与” 结果可签核” 之间尚存在一整套证据工程的距离，其工作量往往超过设计本身。

设计对象并不复杂：五级流水线、两级 Cache、一个置换器与数条定制指令，均属教科书范畴内的结构。实际耗时最多的部分，是那些形式上属于” 记录” 的工作：对 33 组实现策略逐一运行，方能确定策略增益的上界；对 156 个失败端点逐点归因，方能支撑” 仅需改造三处结构” 的判断；将串口原始字节连同 `.XDat` 一并归档，方能在数值受到质疑时即时提供可复核的证据；将 `not_measured` 如实列入正文，方能使” 已完成的八项” 具备可信度。这一认识改变了本团队对” 完成” 的界定：交付物不是设计文件，而是设计文件与其证据链的并集。

第二点体会涉及量化方法。FPGA 平台上的周期加速比极易被无意识地放大：理想存储模型下的仿真会低估软件实现约 20%；以核本体延迟替代端到端会虚高约 6 倍；以 PC 端实现作为分母则失去工程意义。本文最终采用 “仅报板级、仅报端到端、仅对同一份固件” 的准则，数值因此由 200× 量级降至 30.2×，但每一条均可被独立复算。该取舍在评审语境下并不构成不利：可复核的 30 倍较不可复核的 200 倍更具说服力。

第三点体会涉及协作与口径管理。本项目 RTL 与后端工作由不同成员（及辅助工具）并行推进，最易出现的问题并非技术分歧，而是口径歧义——同一个 `+0.087 ns` 在三份文档中可能对应三个源集。若不强制”报数附带 run 名称与源集哈希”，技术讨论将迅速失去共同基准。`docs/` 目录下那些形式冗长的”证据索引”文档，其价值正在于此：它们不是流程负担，而是多人、多工具协作条件下维持结论一致性的基础设施。

第四点体会涉及验证的次序。本项目在板级测试之前先完成 host 侧标准一致性与模块级 golden 比对，使板上出现异常时可迅速排除算法错误，将排查范围收敛至时序与握手。PUF 子系统的首次板级失败即为典型例证：读数显示 `quality = 0`、`mismatch = 1`，而根本原因为上电自动 reconstruct 与固件手动 enroll 争用同一请求，与 RO 特性无关。若无前序仿真结论作为参照，该现象极易被误判为 PUF 质量缺陷，进而导致整份报告结论失真。

第五部分参考文献

1. National Institute of Standards and Technology, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices*, NIST SP 800-232, Aug. 2025. DOI: 10.6028/NIST.SP.800-232.
2. C. Dobraunig, M. Eichlseder, F. Mendel, M. Schläffer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptographic Engineering*, 2021.
3. Ascon C reference implementation, GitHub `ascon/ascon-c`, commit `446347f2`（路径 `crypto_aead/asconaead128/ref/`，许可 CC0-1.0）.
4. M. Meichlseder, *pyascon*, GitHub, commit `ed24e54a`（第二参考实现）.
5. NIST, *Status Report on the Final Round of the NIST Lightweight Cryptography Standardization Process*, NIST IR 8369, 2023.
6. RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA* 与 *Volume II: Privileged Architecture*.
7. AMD/Xilinx, *Zynq-7000 SoC Technical Reference Manual*, UG585.
8. AMD/Xilinx, *Vivado Design Suite User Guide: Implementation*, UG904.
9. AMD/Xilinx, *Vivado Design Suite User Guide: Using Constraints*, UG903.
10. AMD/Xilinx, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, UG906.
11. AMD/Xilinx, *7 Series FPGAs Configuration User Guide*, UG470（Device DNA 与 `DNA_PORT`）.
12. G. E. Suh, S. Devadas, "Physical Unclonable Functions for Device Authentication and Secret Key Generation," *Proc. DAC*, 2007.
13. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 创新设计赛道《选题指南（AMD 自主选题初级组）》.

论文（含其 FPGA 器件、频率、面积与吞吐口径），以及 CoreMark、Dhrystone 类基准的官方说明（若正文引用）。参考文献上限 20 篇，当前 13 篇，余量用于上述两类。

附录 A　术语与缩写

缩写	全称与含义
AEAD	Authenticated Encryption with Associated Data，认证加密
AD	Associated Data，关联数据（不加密但参与认证）
ISE	Instruction Set Extension，指令集扩展（本文特指 custom-0 上的 Ascon 指令）
MMIO	Memory-Mapped I/O，存储器映射访问
KAT	Known-Answer Test，已知答案测试
IP	Intellectual Property，此处指 Vivado IP 核
PUF	Physical Unclonable Function，物理不可克隆函数
RO	Ring Oscillator，环形振荡器
DNA	Device DNA，Xilinx 芯片 57-bit 唯一标识
KDF	Key Derivation Function，密钥派生函数
WNS / TNS	Worst Negative Slack / Total Negative Slack
WHS / THS	Worst Hold Slack / Total Hold Slack
DRC	Design Rule Check
OOC	Out-of-Context，IP 独立综合
COE	BRAM 初值文件（Vivado `blk_mem_gen` 格式）
mcycle	RISC-V M 模式周期计数器 CSR（`0xb00` 与 `0xb80`）
warm / cold	指令已在或不在 I-cache 的执行状态

附录 B　地址映射与指令编码

B.1　SoC 地址映射

窗口	基址	从设备	备注
MEM	0x1000_0000	AXI BRAM Controller → Block RAM（128 KiB）	代码与数据统一空间，可 Cache
CLINT	0x1200_0000	clint.v	msip（+0x0000）、mtimecmp（+0x4000/4）、mtime（+0xbff8/c）
UART	0x2000_0000	uart_ctrl.v	+0x00 status、+0x04 RX/TX、+0x08 ctrl
GPIO	0x2001_0000	gpio.v	+0x00 LED、+0x04 按键事件（读清）
ASCON	0x3000_0000	ascon_mmio.v	寄存器表见 §2.3.12；密钥窗口位于 +0x80 段，见 §2.3.16
CPI	0x6000_0000	cpi_cnt.v	inst、stall_ld、stall_md、flush_br 各 64-bit

B.2　Ascon 定制指令编码（RV32，custom-0 主操作码）

```
 31        25 24    20 19    15 14   12 11     7 6          0
+------------+--------+--------+-------+--------+------------+
|  funct7=0  |  rs2   |  rs1   |funct3 |   rd   |  opcode    |
+------------+--------+--------+-------+--------+------------+
                                                  0001011  (0x0b)

funct3 = 000  ascon.w    state[32*rs2[3:0] +: 32] <- rs1      (1 cycle)
funct3 = 001  ascon.rd   rd <- state[32*rs1[3:0] +: 32]      (1 cycle)
funct3 = 010  ascon.p12  state <- P12(state)                 (multi-cycle)
funct3 = 011  ascon.p8   state <- P8(state)                  (multi-cycle)
```

状态字映射为 w0/w1 = s0 低/高，w2/w3 = s1，w4/w5 = s2，w6/w7 = s3，w8/w9 = s4。非法索引（不小于 10）读返回零、写被忽略，不产生非法指令异常。

B.3　MMIO 加速器寄存器（节选）

完整寄存器表见 §2.3.12：CTRL(0x00)、STATUS(0x04)、ADLEN(0x08)、MLEN(0x0C)、KEY(0x10–1C)、NONCE(0x20–2C)、TAGIN(0x30–3C)、AD_WDATA(0x40)、MSG_WDATA(0x44)、OUT_RDATA(0x48)、TAGOUT(0x4C–58)，以及密钥窗口（0x80–0xD0）。

B.4 PYNQ AXI-Lite 窗口（PS 侧）

窗口	基址	模块	内容
CONTROL	0x4000_0000（4 KiB）	pynq_ctl.v	CTRL（SOC_RESET、BUTTON）、STAT（LED、RST）、ID（0x514E_5950）、VERSION
MAILBOX	0x4000_1000（4 KiB）	pynq_mbox.v	TXDATA、RXDATA、TXSTAT、RXSTAT、RXCLR、TXFLUSH、CNT、ID（0x4D42_4F58，大端读作”MBOX”，小端读作”XOBM”）

两窗口均由 PS7 GP0 以 4 KiB 粒度映射，FCLK0 = 100 MHz；寄存器逐项定义见 §2.3.22。

附录 C　证据索引

本文全部数值的出处按章列示如下。

章	主要证据文件
§1 概述	docs/freeze_checkpoint_20261027.md §1–2；docs/ascon_standard_baseline.md §1、§4
§2 CPU	src/rtl/*.v（defines.v、dcache.v、icache.v、branch_predictor.v、ctrl.v、bus_interconnect.v）；docs/interfaces.md
§3 三实现	src/rtl/ascon/ascon_mmio.v；src/rtl/ascon/ascon_ise.v；docs/ascon_key_management.md；docs/ascon_standard_baseline.md §2
§4 实现	docs/ascon_ppa_comparison.md §1、§3；docs/io_timing_signoff.md §2–§6、§12；docs/timing_acb_validation.md；docs/freeze_checkpoint_20261027.md §2–§6
§5 验证	docs/ascon_standard_baseline.md §3–§5；data/ascon/results/*；project_log/actual_progress.md 2026-10-25/26/27
§6 性能	docs/freeze_checkpoint_20261027.md §3；docs/ascon_ppa_comparison.md §2
§7 PUF	docs/key_subsystem.md；data/ascon/results/board/uart_20261027_keys3.txt；docs/freeze_checkpoint_20261027.md §9
§8 PS/PL	board/vivado_zynq7000_xc7z020_pynq/validation_20261028.md、readme.md、handoff_20261028.md；src/rtl/pynq/*.v；src/pynq/*.py；src/firmware/pynq_svc/*；data/pynq/board_20261028/*
全篇	project_log/actual_progress.md；docs/decisions.md；docs/risks.md

复算入口：build/check_io_signoff.py、build/collect_io_sta_results.py、build/fill_signoff_doc.py、sim/board/run_tests.py、sim/ascon/run_validation.py。